



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali
Laurea Magistrale in Data Science, Calcolo Scientifico &
Intelligenza Artificiale

Master Thesis

GRAPH NEURAL NETWORKS FOR MONEY
LAUNDERING DETECTION

MARTINO PAPA

Advisor: Dott. Daniele Castellana

Academic Year 2025-2026

ABSTRACT

CONTENTS

1	Introduction	7
1.1	Structure of the Dissertation	8
2	Background	9
2.1	Anti Money Laundering theory	9
2.1.1	Financial Crime and Money Laundering	9
2.1.2	Ethics, Privacy and Regulations	12
2.1.3	Traditional AML methods	15
2.1.4	The Growing Challenge	16
2.1.5	Task description	16
2.2	Graph Neural Networks for AML	17
2.2.1	Graph Neural Networks definition	18
2.2.2	Graph Attention Networks	19
2.2.3	Graph Isomorphism Networks	20
2.2.4	Limitations of Local Message Passing	21
3	Methodology	25
3.1	Group-Aware Graph Neural Networks	25
3.1.1	Community-Centric Encoder	26
3.1.2	Extended Markov Random Field Layer	26
3.1.3	Node-Level Classification	27
3.1.4	Group Representation Layer	28
3.1.5	Group-Level Encoding and Prediction	28
3.1.6	Multi-Task Loss	29
3.2	Dataset description	29
3.2.1	Features analysis	31
3.2.2	Building the Financial Graph	36
3.3	Experimental Settings	37
3.3.1	Data Preparation	37
3.3.2	Train, Patience, Validation and Test Split	37
3.4	Model Selection and Training settings	39
3.4.1	Optimizer	39
3.4.2	Subgraph Sampling	40
3.4.3	Early Stopping	40
3.4.4	Hyperparameters	40
4	Results	45
4.1	Hyperparameter Selection Results	45

2 Contents

4.1.1	Loss Function Hyperparameters	45
4.1.2	Architectural Hyperparameters	46
4.2	Models Results	49
4.2.1	Evaluation Metrics	49
4.2.2	General considerations	50
4.2.3	Binary Classification task	52
4.2.4	Multiclass Classification task	53
4.3	Limitations	66
5	Conclusions	67
5.1	Future Directions	67

LIST OF FIGURES

Figure 1	Money Laundering patterns	11
Figure 2	Overview of AMLworld simulation [2]	14
Figure 3	Non-isomorphic graphs that are not discriminated by the 1-WL test	22
Figure 4	IBM AMLSim datasets statistics [2]	30
Figure 5	IBM AMLSim account records	31
Figure 6	IBM AMLSim transaction records	32
Figure 7	IBM AMLSim patterns examples	33
Figure 8	IBM AMLSim patterns distribution over fraudulent edges	34
Figure 9	IBM AMLSim amounts distribution	34
Figure 10	IBM AMLSim currencies distribution	35
Figure 11	IBM AMLSim payment formats distribution	35
Figure 12	IBM AMLSim hour and day of the week distributions	36
Figure 13	IBM AMLSim banks distribution	36
Figure 14	Training and patience loss for GAGNN-GAT (Binary task)	53
Figure 15	Confusion matrix for GAGNN-GAT (Binary task)	54
Figure 16	Evaluation report for GAGNN-GAT (Binary task)	54
Figure 17	Training and patience loss for GAGNN-GIN (Binary task)	55
Figure 18	Confusion matrix for GAGNN-GIN (Binary task)	55
Figure 19	Evaluation report for GAGNN-GIN (Binary task)	56
Figure 20	Training and patience loss for GAT baseline (Binary task)	56
Figure 21	Confusion matrix for GAT baseline (Binary task)	57
Figure 22	Evaluation report for GAT baseline (Binary task)	57
Figure 23	Training and patience loss for GIN baseline (Binary task)	58
Figure 24	Confusion matrix for GIN baseline (Binary task)	58
Figure 25	Evaluation report for GIN baseline (Binary task)	59
Figure 26	Training and patience loss for GAGNN-GAT (Multiclass task)	60

4 List of Figures

Figure 27	Confusion matrix for GAGNN-GAT (Multiclass task)	60
Figure 28	Evaluation report for GAGNN-GAT (Multiclass task)	61
Figure 29	Training and patience loss for GAGNN-GIN (Multiclass task)	61
Figure 30	Confusion matrix for GAGNN-GIN (Multiclass task)	62
Figure 31	Evaluation report for GAGNN-GIN (Multiclass task)	62
Figure 32	Training and patience loss for GAT baseline (Multiclass task)	63
Figure 33	Confusion matrix for GAT baseline (Multiclass task)	63
Figure 34	Evaluation report for GAT baseline (Multiclass task)	64
Figure 35	Training and patience loss for GIN baseline (Multiclass task)	64
Figure 36	Confusion matrix for GIN baseline (Multiclass task)	65
Figure 37	Evaluation report for GIN baseline (Multiclass task)	65

"Citation"
— *citation's author*

INTRODUCTION

Money laundering is the criminal process of disguising or concealing the origin, identity, and ownership of illegally acquired funds. Its detection has become a foundational pillar of modern economic security. The rapid digitization of financial services has opened new possibilities for fraudulent activities. Undetected ML leads to massive economic losses for individuals and institutions, undermines the public trust in the financial system and frequently serves to finance criminal organizations. The scale of this phenomenon is staggering: the United Nations Office on Drugs and Crime estimates that between 2% and 5% of global GDP is laundered annually, with less than 1% of these illicit financial flows being successfully detected [19, 9]. According to the Canadian Anti Fraud Centre, 704 million dollars were lost because of frauds in 2025 only, confirming the raising trend of the previous years [3]. Consequently, developing robust mechanisms to identify and stop it is critically important to preserving both market integrity and social stability.

The goal of this dissertation is to evaluate the effectiveness of Graph Neural Network (GNN) based models on detecting money laundering patterns. Both precision and explainability will be taken into account and we will propose two different tasks; a binary classification to label transactions as licit or illicit, and a multiclass classification to detect the specific type of laundering strategy employed which, in case of success, will enhance the explainability of the model.

In particular we will focus on the Group Aware Graph Neural Network (GAGNN) architecture proposed in [5]. GAGNN is a GNN-based model that leverages a multi-component loss function to predict if a transaction is licit or illicit. There are a total of 3 losses, a node loss, an edge loss and a group loss. The group one is the novelty of the proposal since it introduces the concept of community by grouping nodes that are suspected to be doing money laundering together. More details on

the architecture will be provided in Section 3.1. We will investigate the following inquiries;

- whether the GAGNN architecture outperforms other baseline models on the binary task;
- whether the GAGNN architecture could be improved by enhancing Graph Isomorphism Network (GIN) instead of Graph Attention Network (GAT);
- whether the GAGNN architecture could perform well also on the multiclass task.

1.1 STRUCTURE OF THE DISSERTATION

BACKGROUND

2.1 ANTI MONEY LAUNDERING THEORY

2.1.1 *Financial Crime and Money Laundering*

Understanding how money laundering works is fundamental to guarantee both an effective and explainable detection of suspicious transaction. This section offers a theoretical introduction to money laundering providing a mathematical formulation of the problem together with a summary of common strategies employed by criminals to avoid detection.

The Limitations of the Lonely Scammer

Individual scammers typically rely on simple and intuitive techniques to introduce illicit funds into the legitimate economy. One of the most prevalent methods is known as “smurfing”. In simple terms, smurfing involves breaking down a large sum of illicit money into many smaller, inconspicuous chunks to deposit them without raising suspicion.

More formally, individual scammers rely on smurfing to avoid Anti-Money Laundering (AML) detection mechanisms, which usually flag transactions above a specific threshold $\tilde{x}$. By splitting their initial illicit funds x into smaller transactions x_n , such that each transaction satisfies $x_n \leq \tilde{x}$, they minimize the risk of detection [7]. The mathematical expectation of profit π_s for a scammer engaging in smurfing can be defined as:

$$\pi_s = x(1 - \nu) \tag{2.1}$$

where x represents the total initial illicit funds, and $\nu \in (0, 1)$ is the friction cost or commission proportion associated with the smurfing process (e.g., the fee paid to mules).

While smurfing is effective for small amounts, its scalability is severely limited. When attempting to launder much larger volumes of funds, an individual scammer operating from a single account faces a significantly higher probability of detection $(1 - p)$, where p is the probability of successfully avoiding detection. If caught, the scammer is subjected to a severe monetary penalty T , which includes both the total confiscation of the funds and associated judicial fines [7]. Therefore, operating as a “lonely scammer” becomes mathematically and practically unviable for large-scale operations due to the massive expected cost of $(1 - p)T$.

Fraud Rings

In regard to overcome those issues, criminals have improved their techniques by employing highly sophisticated obfuscation tactics. Instead of acting alone, they form complex “fraud rings” to distribute illicit funds across vast networks of temporary accounts. To further break the traceable link between sender and receiver, illicit actors frequently use cyclic transfers and cryptocurrency **mixers** services that pool together funds from multiple users, mix them, and redistribute them at random intervals and amounts. These advanced tactics create dense, highly camouflaged transaction topologies that easily evade threshold-based rules and simple heuristic detection methods.

To achieve their goal they utilize a strategy of transaction layering, often characterized by a topological depth L , and network diversification to obscure the money trail [7, 15]. By spreading the funds through a complex topology, the fraud ring successfully increases its non-detection probability p_L , which now depends explicitly on the layering depth L . The expected profit of a fraud ring utilizing L layers can be mathematically expressed as:

$$E[\pi_L] = p_L x - (1 - p_L)T - c(L) \quad (2.2)$$

where $E[\pi_L]$ is the expected profit, p_L is the enhanced probability of evading detection when L layers are used, x is the total illicit funds, T is the total penalty if caught, and $c(L)$ represents the monotonic cost function associated with establishing and maintaining the L transaction layers. Because the fraud ring dramatically increases the probability of evading detection ($p_L \rightarrow 1$ as L increases), the expected payoff remains highly profitable even when factoring in the complex layering costs $c(L)$ [7].

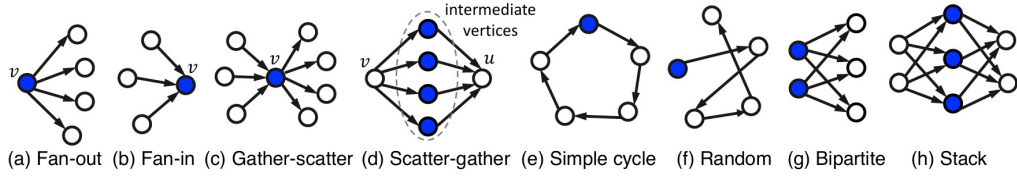


Figure 1: Money Laundering patterns

Money Laundering patterns

Understanding the structural patterns that criminality assumes is fundamental to develop effective detection models and to employ strategies to enhance the explainability of them. According to the literature, there are 8 different patterns that are often used in money laundering [18, 2]. Which are:

- *fan-out*, shown in Figure 1 a, characterized by outgoing edges from a node v to more than 2 adjacent nodes;
- *fan-in*, Figure 1 b, v is connected to more than 2 nodes by incoming edges;
- *gather-scatter*, Figure 1 c, is the combination of a fan-in and a fan-out pattern on the same node v ;
- *scatter-gather*, Figure 1 d, also composed by a fan-in and a fan-out pattern but on two different nodes, sharing the same intermediate neighbors;
- *simple-cycle*, Figure 1 e, characterized by a sequence of transactions that starts and ends at the same node;
- *random*, Figure 1 f, similar to the cycle but funds do not return to the original account;
- *bipartite*, Figure 1 g, moves funds from a set of original nodes to a set of target nodes, with no transactions between accounts of the same set;
- *stack*, Figure 1 h, extends the bipartite pattern by adding a second bipartite layer.

It's important to mention that in some cases some patterns can be found in both illicit and licit transactions [23]. Specifically, *fan-in* and *fan-out*

are patterns that are also frequent in legitimate, every-day transactions, such as salary payments or transfers between family members or friends. That being said, the detection of such patterns remains a fundamental component of any money laundering detection models, albeit insufficient on its own. Furthermore, a model that correctly classifies those patterns provides a much more interpretable result than one that simply performs a binary classification between licit and illicit transactions. In Section 2.1.2 we will see why explainability is of fundamental importance in the financial field.

2.1.2 *Ethics, Privacy and Regulations*

In this section we will begin by introducing a brief overview of the regulations that currently govern the field of AML. Subsequently a discussion on the main issues that the use of machine learning models may introduce will be provided, together with suggested solutions to address them. A special focus will be reserved to the issues of explainability and privacy. To conclude we will consider the use of Synthetic Datasets as a possible solution to these issues, analyzing their limits and advantages in the context of AML.

AML Global Regulations

Effective money laundering detection requires a combination of domestic laws, regulatory oversight, and international cooperation. Since the end of the 20th century the United States and the European Union as key jurisdictions, together with the FAFT¹ as the global leading standard-setter have been developing and elaborate framework for AML [1].

In 1989 the G-7 nations established an inter-governmental task force to develop and promote international guidelines in regard to fight money laundering, the FATF. Nowadays FAFT still holds a central role in the AML field. It has issued 40 recommendations for national laws and international cooperation [1]. These principles have been adopted by over than 200 jurisdictions and they fall into 5 main areas [16]:

- Risk Based Approach (RBA): jurisdictions must identify and understand the financial crime risks that they face, allocating the right amount of resources to mitigate them;

¹ The Financial Action Task Force (FATF) is an intergovernmental policymaking body established in 1989 by the G7

- Legal Framework: members nations must adopt laws that criminalize money laundering; furthermore authorities must be given the power to freeze and confiscate funds when needed;
- Preventive Measures:
 - Customer Due Diligence (CDD): financial institutions must verify the identity of their customers;
 - Ultimate Beneficial Ownership (UBO): company's owners must be verified, this addresses the issue of criminals hiding behind shell institutions;
 - Record Keeping: financial institutions must maintain their transactions records for at least 5 years and file Suspicious Transactions Reports (STRs) to Financial Intelligence Units (FIUs).
- Supervision and Transparency: regulators must actively supervise financial institutions; members nations must immediately enforce UN financial sanctions when required to.
- International cooperation: members must quickly share information and provide mutual legal assistance in international investigations.

In regard to those guidelines FAFT keeps a *black-list* and a *grey-list* of jurisdictions that are not compliant with its standards [1].

Ethical and practical issues

When considering real-world datasets, models that detect financial crime are trained on sensitive data that needs to be protected. To avoid the leakage of sensitive information, before feeding the data to machine learning models, it is necessary to apply privacy-preserving techniques. The most common practice is to apply differential privacy, which adds noise to the dataset to protect individual data points. That noise can be quantified using the privacy budget ϵ , which determines the trade-off between privacy and utility. All this comes at the cost of reducing the accuracy of the model.

Another significant issue to consider is explainability. Regulators require banks to explain their decisions, and this requirement extends to the machine learning models they use. Deep Learning models, albeit generally more precise and powerful than traditional machine learning models,

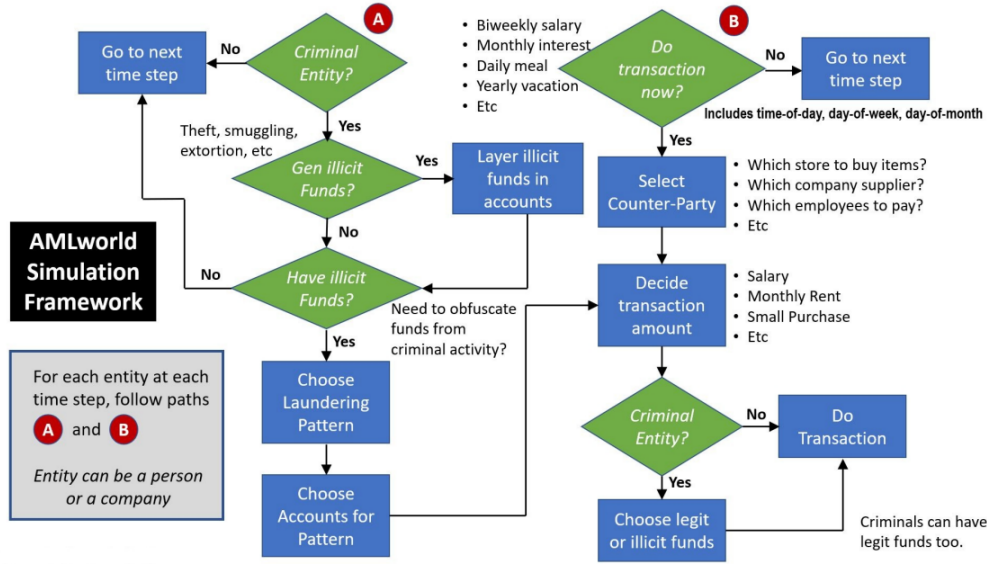


Figure 2: Overview of AMLworld simulation [2]

are often *black-boxes*, meaning their decision process is very difficult to understand for a human being.

Real datasets are also often limited by the fact that the informations are not shared accross jurisdictions, making it difficult to detect frauds that cross borders.

To tackle all those problematics researchers at IBM developed a simulator for generating realistic financial transactions, called AMLWorld (2023) [2]. This simulator solves the privacy problem, simulates multi-jurisdictional transactions and it even tries to takle the explainability issue by providing as lable not only a boolean value for the presence of a fraud, but also the pattern that caracterized it. This wouldn't be possible in real life, but models trained on this synthetic dataset could be able to generalize the detection of patterns even in real world scenarios, highly enhancing interpretability. Figure 2 provides an overview on how AML-World generates realistic financial transactions and patterns of financial crime.

The possible laundering patterns are the one described in Section 2.1.1; the model assumes the existence of 9 different surces of criminal activities: extortion, loan sharking, gambling, prostitution, kidnapping, robbery, embezzlement, drugs, and smuggling. The amount of money and the frequency of these criminal enterprises vary by both activity category and acting entity [2]. The funds are then inserted in the financial system alongside the legit ones.

2.1.3 *Traditional AML methods*

Anti Money Laundering is a critical component of the global financial system. Financial institutions ought to implement robust mechanisms to comply with regulatory standards and protect the integrity of the financial system [16].

Rule-based models have long been employed to detect suspicious transactions. These models consist of a set of predefined rules that are used to flag transactions that deviate from typical financial behavior. The criteria are typically based on regulatory requirements, historical data, and known patterns of illicit behavior. For example we could decide to flag transactions that exceed a certain amount, that rapidly transfer funds between accounts or that involve a high-risk jurisdiction.

The main advantage of this approach is its transparency and interpretability. Furthermore they are computationally inexpensive, easy to implement and they can operate in real-time [16]. Unfortunately there are also several limitations. They tend to produce a high amount of false positives, which leads to more work from human operators that will have to deal with the flagged transactions. They could be easily exploited by criminals who are aware of the rules and can tailor their illicit activities to avoid detection. This will be further discussed in Section 2.1.1.

Due to its tendency of producing a huge amount of false positives and the huge amount of data that nowadays is produced by financial institutions, the rule-based approach is becoming less and less effective.

At the same time big datasets allow the usage of data-driven solutions that can learn directly from historical data. First attempts in this direction involved standard machine learning approaches such as logistic regression and support vector machines [4]. Those models improved the state of the art but they still struggled to detect human brain-armed patterns due to their very limited parameter capacity [5].

For this reason the research field shifted toward deep learning solutions which offer a much wider and complex hypothesis space, allowing to capture high order relationships between data, at the cost of a much worse interpretability. Nowadays the challenge is to combine high performance together with explainability to allow the usage of effective models in the highly regulated financial field, a necessity increasingly emphasized by both financial institutions and supervisory authorities [14]. We will discuss this further in Section 2.1.2

2.1.4 The Growing Challenge

Detecting fraud is becoming increasingly difficult. The sheer volume and velocity of digital transactions make manual oversight impossible and heavily strain classical machine learning architectures. Furthermore, the rise of decentralized finance (DeFi), Web3, and cryptocurrencies has introduced a paradigm of pseudo-anonymity that complicates traditional tracing.

On top of that, criminals are constantly developing more and more sophisticated methods to avoid being caught. As Cuèllar (2003) noted, *"raising the marginal cost of perpetrating criminal activity can itself conceivably have counterintuitive effects. To the extent that criminals engaged in crimes for profit are acting rationally, then raising the cost of crime can discourage smaller operations and clear the market allowing more sophisticated criminal organizations to innovate and take more control from less organized criminals"* [8]. Hence, to prevent inadvertently incentivizing sophisticated criminal networks, detection mechanisms must be updated continuously. The framework proposed in this study leverages deep learning models that support online updates. This ensures that the detection algorithm remains aligned with the most recent transaction patterns.

2.1.5 Task description

In this dissertation we consider the problem of Edge-Level money laundering detection. This means, considering the Graph $G = (V, E)$ as in Definition 2.1, trying to learn a function f that, for each transaction $e \in E$, predicts its class. In particular we will consider two different settings. A binary classification where each edge is labeled as fraudulent or legit and a multi-class classification where, in case of illicit transaction, we also predict the type of pattern, amongst the one described in Section 2.1.1, it's involved in.

Formally we seek a function f that minimizes the error $L(f)$ over a distribution $\mathcal{D}$ of Graphs.

$$\operatorname{argmin}_{f \in \mathcal{H}} \{L(f) \doteq \mathbb{E}_{(x,y) \sim \mathcal{D}} [l(f(x), y)]\} \quad (2.3)$$

Practically there's no way to know $\mathcal{D}$. Instead we will consider a dataset $G_D = (V_D, E_D)$, where both nodes and edges are equipped with feature

vectors and we will seek a function f that minimizes $L(f)$ over this dataset computing the empirical risk:

$$\operatorname{argmin}_{f \in \mathcal{H}} \left\{ \hat{L}(f) \doteq \frac{1}{|E_D|} \sum_{e \in E_D} l(f(x_e), y_e) \right\} \quad (2.4)$$

This is just the theoretical overview of the problem. In Chapter 3 we will provide more insights on how to define the hypothesis space $\mathcal{H}$ and the loss function l .

As mentioned in Section 2.1.2, the strict privacy regulations that governs the financial world, makes it hard to find real-world datasets. In this dissertation, for the reasons explained in Section 2.1.2, we will use the synthetic dataset *IBM AMLSim*. This is perfect for our scope, since it provides edge labels for both binary and multiclass classification.

2.2 GRAPH NEURAL NETWORKS FOR AML

The pervasive digitisation of financial services has generated transaction ecosystems of unprecedented scale and complexity. Classical machine learning approaches, whether logistic regression, support vector machines, or shallow neural networks, operate on tabular feature vectors and treat each data point as an independent, identically distributed observation. This assumption is fundamentally at odds with the relational nature of financial data, where the behaviour of an account cannot be understood in isolation but only in the context of the network of counterparties it interacts with. Graph Machine Learning (GML) addresses this gap by enabling models to operate directly on graph-structured data, learning representations that jointly encode both the attributes of individual entities and the topology of their interactions [11].

The field of GML encompasses a broad spectrum of methodologies. Early approaches relied on hand-crafted structural descriptors, such as degree centrality, clustering coefficients, and motif counts, that were pre-computed and then fed into standard classifiers. A more principled family of methods, known as graph kernels, extended the idea of kernel machines to structured data by defining similarity functions over pairs of graphs or node neighbourhoods [13]. While both families have proven useful, they suffer from a common limitation: the feature extraction process is decoupled from the learning objective, preventing the model from adapting its graph representations to the task at hand.

Modern GML overcomes this limitation through end-to-end learned representations. These methods are typically organised around three levels of prediction granularity. *Node-level* tasks assign a label or a continuous value to each vertex; canonical examples include community detection and entity classification. *Graph-level* tasks predict a property of an entire graph, as in molecular property prediction or graph classification. *Edge-level* tasks, finally, operate on pairs of nodes connected by an edge and include both link prediction, that is inferring which connections are likely to form, and edge classification, which is precisely the setting adopted in this dissertation for the detection of fraudulent transactions.

Across all these tasks, Graph Neural Networks (GNNs) have emerged as the dominant paradigm. Their success stems from a principled inductive bias: rather than learning a fixed function of global graph statistics, GNNs propagate and transform information along the edges of the graph through an iterative message-passing mechanism. This endows them with the ability to produce node representations that are sensitive to local neighbourhood structure while remaining equivariant to node permutations, a property that no tabular model can achieve. Furthermore, parameters are shared across all nodes and edges, making GNNs naturally scalable to graphs with millions of vertices. When node and edge attributes are available, as is the case in the financial graphs considered here, GNNs can seamlessly fuse topological signals with rich feature information, yielding representations that are far more expressive than either source alone [21].

In the following subsections we formalise the message-passing framework underpinning GNNs and discuss two specific architectures employed in this work, together with their theoretical limitations in the context of AML detection.

2.2.1 Graph Neural Networks definition

The necessity of capturing complex interactions between accounts and transactions led to the idea of modeling the financial feed as a graph, where accounts are nodes and transactions are edges [17].

Definition 2.1 (Financial Graph). *A financial graph is a directed graph $G = (V, E)$, where V represents a set of financial entities (e.g. bank accounts or individuals), and E represents a set of transactions between these entities. Each edge $e \in E$ is directed from the sender to the receiver and may be associated with features.*

GNNs rely on message passing to aggregate information from neighbors and learn representations of the nodes that will be given as input to a classifier or regressor to predict the probability that an account (Node-Level detection), a transaction (Edge-Level detection) or a sub-graph (Graph-Level detection) is fraudulent. These methods allow to capture complex relationships, learn features automatically, handle dynamic graphs, and integrate multimodal data.

Formally the message passing process at the k -th layer in a GNN is defined as follows:

$$\begin{aligned} m_i^{(k)} &= \text{AGGREGATE} \left\{ \text{MSG} \left(h_j^{(k-1)}, e_{ji} \right) : j \in \mathcal{N}(i) \right\} \\ h_i^{(k)} &= \text{UPDATE} \left(h_i^{(k-1)}, m_i^{(k)}, x_i \right) \end{aligned} \quad (2.5)$$

where $\mathcal{N}(i)$ denotes the set of neighboring nodes of node i , e_{ji} is the feature vector of the edge from node j to node i , MSG (e.g. MLP) is the function that merges informations from the neighbor and the connecting edge and x_i is the feature vector of node i . Sometimes edge features are not used during the message passing process, in such cases $m_i^{(k)}$ is defined as:

$$m_i^{(k)} = \text{AGGREGATE} \left\{ h_j^{(k-1)} : j \in \mathcal{N}(i) \right\} \quad (2.6)$$

After K iterations of message passing, the resultant node embeddings $H^{(K)}$ are then used to execute the designated task, in our case the detection of fraudulent accounts [6].

There are different types of GNNs; defined by how the UPDATE and AGGREGATE functions are implemented.

2.2.2 Graph Attention Networks

As mentioned in Section 2.2 different types of GNNs are defined by different AGGREGATE and UPDATE functions.

Graph Attention Networks introduce learned attention coefficients to improve the fixed weight aggregation of classical graph convolutions, allowing the model to understand which neighbours are more relevant towards the classification task [20]. The AGGREGATE function is defined as:

$$m_i^{(k)} = \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k,t)} W^{(k,t)} h_j^{(k-1)} \quad (2.7)$$

where H is the number of attention heads, $\mathcal{N}(i)$ is the set of neighbors of node i , $W^{(k,t)}$ is the weight matrix for the attention head t at layer k and $\alpha_{ij}^{(k,t)}$ is the attention coefficient between node i and its neighbor j at layer k and head t defined as

$$\alpha_{ij}^{(k,t)} = \frac{\exp(e_{ij}^{(k,t)})}{\sum_{l \in \mathcal{N}(i)} \exp(e_{il}^{(k,t)})} \quad (2.8)$$

where, in our case, the unnormalized attention scores $e_{ij}^{(k,t)}$ also rely on the edge features ℓ_{ij} of the edge (i, j) .

$$e_{ij}^{(k,t)} = \text{LeakyReLU} \left(\mathbf{a}^{(k,t)\top} \begin{bmatrix} W^{(k,t)} \mathbf{h}_i^{(k-1)} & W^{(k,t)} \mathbf{h}_j^{(k-1)} & W_e^{(k,t)} \ell_{ij} \end{bmatrix} \right) \quad (2.9)$$

where $\mathbf{a}^{(k,t)}$, $W^{(k,t)}$ and $W_e^{(k,t)}$ are learnable parameters.

The UPDATE function is then defined as:

$$\mathbf{h}_i^{(k)} = \begin{cases} \text{ReLU}(\mathbf{m}_i^{(k)}) & k < L \\ \text{Id} \left(\frac{1}{H} \sum_{t=1}^H \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k,t)} W^{(k,t)} \mathbf{h}_j^{(k-1)} \right) & k = L \end{cases} \quad (2.10)$$

Notice how the UPDATE function is different for the last layer. This is due to the fact that it averages the single heads outputs to produce an embedding of dimension d_{out} instead of concatenating them; which is the case for intermediate layers.

The embeddings are then leveraged for edge classification. Let (i, j) be an edge, his final representation will be the concatenation of the embedding of the source node $\mathbf{h}_i^{(L)}$, the embedding of the destination node $\mathbf{h}_j^{(L)}$ and the feature vector ℓ_{ij} . $[\mathbf{h}_i^{(L)}, \mathbf{h}_j^{(L)}, \ell_{ij}]$ is then passed through a Multi-Layer Perceptron that computes the prediction.

2.2.3 Graph Isomorphism Networks

Graph Isomorphism Networks achieve the theoretical upper bound of expressive power for message passing GNNs [22]. By using a MLP in the update function instead of a simple linear transformation, they are able to match the expressive power of the 1-Weisfeiler-Lehman test (Definition 2.3). The AGGREGATE function is simply defined as:

$$\mathbf{m}_i^{(k)} = \sum_{j \in \mathcal{N}(i)} \mathbf{h}_j^{(k-1)} \quad (2.11)$$

The UPDATE function, which is what actually differentiate GIN from other GNNs, is defined as:

$$h_i^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \varepsilon^{(k)} \right) h_i^{(k-1)} + m_i^{(k)} \right) \quad (2.12)$$

where $\varepsilon^{(k)}$ is a learnable parameter. In our implementation, we used a two-layer MLP with a ReLU activation:

$$\text{MLP}(z) = W_2 \text{ReLU}(W_1 z + b_1) + b_2 \quad (2.13)$$

where W_1 , W_2 , b_1 and b_2 are learnable parameters. The embeddings are then leveraged for edge classification in the same way as for GAT.

2.2.4 Limitations of Local Message Passing

GATs and GINs, both belonging to the family of local message passing GNNs, share common limitations.

Weisfeiler-Leman test upperbound

Theoretically it has been proved that the expressive power of local message passing GNNs is upper bounded by the 1-Weisfeiler-Leman test [22];

Definition 2.2 (Isomorphic Graphs). *Two graphs $G = (V, E)$ and $G' = (V', E')$ are isomorphic if there exists a bijection $f : V \rightarrow V'$ such that for all $u, v \in V$, $(u, v) \in E \iff (f(u), f(v)) \in E'$.*

Definition 2.3 (1-Weisfeiler-Leman test). *The 1-Weisfeiler-Leman (1-WL) test is an algorithm to determine whether two graphs $G = (V, E)$ and $G' = (V', E')$ are isomorphic or not. It works by iteratively assigning colours to nodes according to the local structure of the graph. The process is defined as follows:*

1. **Initialization:** Every node it's initialised with the same color $c_i^{(0)} = c^{(0)} \forall i \in V, V'$;
2. **Iteration:** At each step k , each node updates its colour by collecting the colours of all its immediate neighbours and combining them with its current colour to create a new one:

$$c_i^{(k)} = \text{HASH} \left(c_i^{(k-1)}, \left\{ \left\{ c_j^{(k-1)} : j \in \mathcal{N}(i) \right\} \right\} \right) \quad (2.14)$$

the HASH function guarantees that different combinations of colours will always produce a new distinct colour.

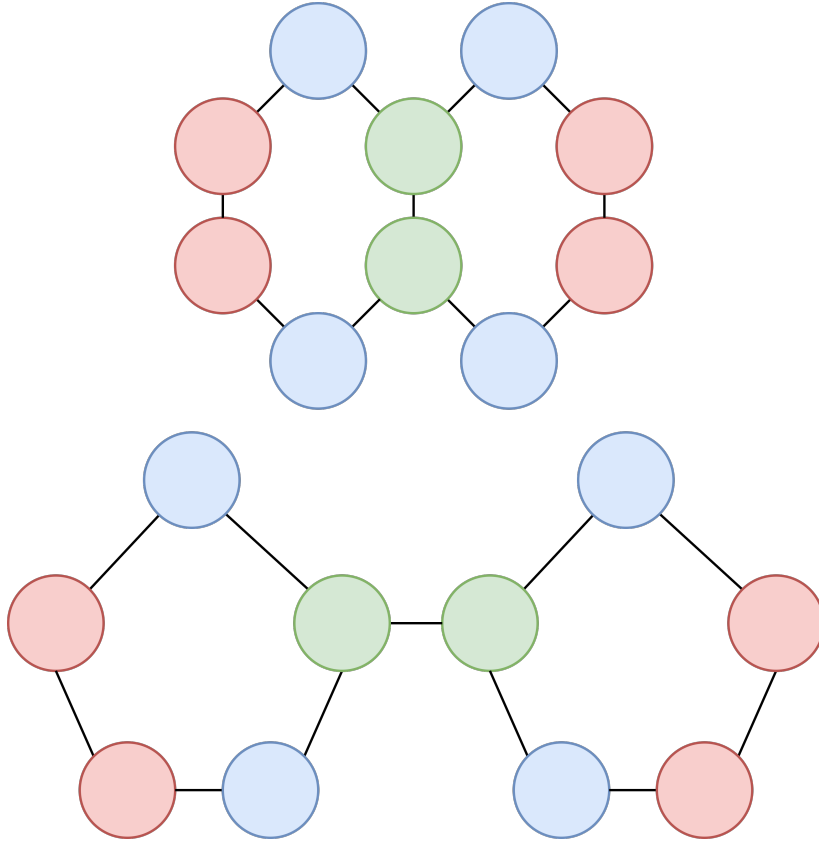


Figure 3: Non-isomorphic graphs that are not discriminated by the 1-WL test

3. **Comparison:** After each step, the algorithm compares the number of nodes for each colour in both graphs. If they are different, we are certain that the graphs are not isomorphic. Unfortunately, if the colours stop updating and the sets are still identical, the test is inconclusive; there is no guarantee that the two graphs are actually isomorphic. In Figure 3 we can see an example of non-isomorphic graphs that are not discriminated by the 1-WL test.

This means that message passing GNNs are unable to discriminate certain non-isomorphic graph structures, limiting their ability to detect global topological patterns.

Oversmoothing

Growing the number of layers L in a GNN generally allows to capture higher order and more complex dependencies. However, as L grows, node embeddings tend to converge to a stationary point that is virtually

identical for all nodes [6]. When this happens, the node embeddings lose their ability to represent different nodes and the model loses its discriminative power.

Oversquashing

To capture interactions between nodes separated by a distance r , GNNs require at least r layers of message passing.

While the receptive field $|\mathcal{N}_r(v)|$ grows exponentially with r , the representation $h_v \in \mathbb{R}^d$ maintains its fixed capacity d . Consequently, propagating signals across multiple hops forces an exponentially increasing volume of neighborhood information into a fixed-size vector, creating an informational bottleneck.

This gets worse across graph bottlenecks, where signals from distant nodes u are heavily diluted and fail to influence the final embedding of v . [6].

To overcome those limits we will leverage Group-Aware GNNs (GAGNNs) [5] (Section 3.1), introducing a clustering mechanism to merge nodes that are suspected to be doing money laundering together and predicting their labels as a groups.

METHODOLOGY

In this dissertation we propose a total of four different models; Graph Attention Networks (GAT, Section 2.2.2) and Graph Isomorphism Networks (GIN, Section 2.2.3) will be used as evaluation baselines; Finally, to mitigate the limitations (Section 2.2.4) of the two previous models, we will consider the architecture proposed in the paper Group-Aware Graph Neural Network [5] (Section 3.1) which, by aggregating nodes that are involved in suspicious transactions into gangs, and running a GNN on the obtained grouped graph, tries to detect money laundering organizations. We will consider two variants of the model one based on a GAT and one based on a GIN.

3.1 GROUP-AWARE GRAPH NEURAL NETWORKS

As introduced in Section 2.1.1, money laundering is inherently an organized crime. Illicit transactions are not isolated events scattered randomly across the graph, but are rather concentrated among clusters of colluding accounts acting as gangs [5]. Detecting fraud solely at the node or edge level tends to ignore this crucial group structure.

To overcome these limitations, we adopt the *Group-Aware Graph Neural Network* (GAGNN) architecture proposed by Cheng et al. [5]. GAGNN is a multi-component pipeline that can embed any of the previously described GNNs (GAT and GIN) as backbone and enriches them with two key mechanisms:

1. An *eMRF layer* (Edge Markov Random Field) that encourages nodes involved in the same fraud ring to receive similar embeddings.
2. A *Group Representation Layer* that explicitly detects and groups suspicious accounts into super-nodes, adding a community-level classification.

The overall GAGNN pipeline proceeds in five steps.

3.1.1 Community-Centric Encoder

The first step employs a standard GNN backbone (GAT or GIN) to transform the raw node feature matrix $X^{(1)} \in \mathbb{R}^{N \times d_{\text{feat}}}$ into enriched embeddings $X^{(2)} \in \mathbb{R}^{N \times d_{\text{out}}}$ by stacking L message passing layers;

$$X^{(2)} = f_{\text{GNN}}^{(L)} \left(\dots f_{\text{GNN}}^{(1)} \left(X^{(1)}, E \right) \dots \right) \quad (3.1)$$

where E denotes the set of edges. A classifier then uses these embeddings to produce a preliminary node-level prediction p_{fraud} , which will be utilized by the subsequent eMRF layer;

$$p_{\text{fraud}} = \sigma \left(W_c X^{(2)} \right) \in [0, 1]^N \quad (3.2)$$

where σ is the sigmoid activation function. Notice that p_{fraud} represents the probability of a node being illicit. Because the node classification step focuses strictly on detecting the presence of laundering, this prediction remains binary even in multiclass scenarios. The differentiation of specific laundering patterns is handled exclusively at the transaction level later in the pipeline.

3.1.2 Extended Markov Random Field Layer

The eMRF layer refines the embeddings $X^{(2)}$ by pulling together nodes that belong to the same community and repelling those that do not. This community similarity is derived from both the graph topology and the node attributes.

First, the topological similarity $\varepsilon(v_i, v_j)$ measures how much more connected nodes v_i and v_j are compared to the average connection density of a random graph:

$$\varepsilon(v_i, v_j) = \frac{d_i d_j}{2|E|} - a_{ij} \quad (3.3)$$

where d_i is the degree of node v_i , $|E|$ is the total number of edges, and a_{ij} is the adjacency indicator; $a_{ij} = 1$ if an edge exists between v_i and v_j and 0 otherwise. Next, the attribute similarity is computed as the normalized cosine similarity between the intermediate embeddings:

$$R_z(v_i, v_j) = \text{Normalize} \left(\frac{h_i^{(2)\top} h_j^{(2)}}{\|h_i^{(2)}\| \|h_j^{(2)}\|} \right) \quad (3.4)$$

These two measures are linearly combined via a trade-off parameter β :

$$\gamma(v_i, v_j) = \beta \varepsilon(v_i, v_j) + (1 - \beta) R_\zeta(v_i, v_j) \quad (3.5)$$

Nodes predicted to belong to the same class by the coarse classifier are attracted, while nodes of different classes are repelled. The pairwise potential $\Psi(v_i, v_j)$ is defined as:

$$\Psi(v_i, v_j) = (1 - 2\sigma(v_i, v_j)) \cdot \gamma(v_i, v_j) \quad (3.6)$$

where $\sigma(v_i, v_j) = \mathbf{1}[\lfloor p_{\text{fraud},i} + 0.5 \rfloor = \lfloor p_{\text{fraud},j} + 0.5 \rfloor]$ is a binary indicator of the same predicted class. Because the coarse node classifier is strictly binary, this indicator remains mathematically identical whether the overarching task is binary or multiclass.

Notice that our implementation differs mathematically from the original paper formulation, which used an exponential sign term $(-1)^{\sigma(v_i, v_j)}$. Since the derivative of a^x relies on $\ln(a)$, computing $\ln(-1)$ is mathematically undefined and causes PyTorch to output NaN gradients during the backward pass. We solved this by using the equivalent linear mapping $(1 - 2\sigma)$. Furthermore, since the hard threshold $\lfloor \cdot + 0.5 \rfloor$ is not differentiable, we utilize a Straight-Through Estimator (STE). During the forward pass, the model uses the hard indicator, while during the backward pass, gradients are propagated through a soft continuous approximation.

Finally, the refined embeddings $X^{(3)}$ are computed using a mean-field update:

$$X^{(3)} = X^{(2)} - \Psi X^{(2)} H^{(2)} \quad (3.7)$$

where $H^{(2)}$ is a learnable weight matrix.

3.1.3 Node-Level Classification

The refined embeddings $X^{(3)}$ are mapped to node-level classification logits by a two-layer Multi-Layer Perceptron (MLP):

$$p_{\text{node}} = \text{MLP}_{\text{node}}(X^{(3)}) \quad (3.8)$$

These logits provide the first of three supervision signals used to train the network.

3.1.4 Group Representation Layer

This layer classifies individual edges and discovers the latent groups of colluding accounts. First, for each directed edge (i, j) , the layer concatenates the source and destination embeddings with the raw edge features to compute the transaction probability:

$$p_{\text{trans}}(e_{ij}) = \text{MLP}_{\text{edge}}([X_i^{(3)} \parallel X_j^{(3)} \parallel \ell_{ij}]) \quad (3.9)$$

Then we used this distribution to construct the communities graph. Considering an edge e_{ij} , the two accounts i and j will be grouped together with a probability equal to their predicted illicit probability $p_{\text{trans}}(e_{ij})$ or, if the edge is in the training set, we will simply use its label to decide whether to group together the two nodes or not. In the case of a multi-class classification task, this Bernoulli probability is computed as $1 - p_{\text{licit}}(e_{ij})$, effectively considering all illicit classes together. The resulting groups are then collapsed into single super-nodes whose feature vector is the mean of its members' original features. Two communities will be connected by an edge if at least one node from one community have an edge with a node in the other community; the edge features will be dropped mantaing only the information about the connectivity.

$$\hat{x}_g = \frac{1}{|\mathcal{G}_i|} \sum_{v \in \mathcal{G}_i} x_v \quad (3.10)$$

Each group $\mathcal{G}$ is considered illicit if it contains at least one illicit account, licit otherwise. Formally we define the group label as:

$$\hat{y}_{\mathcal{G}} = \begin{cases} 1 & \text{if } \exists V \mid V \text{ is ML} \wedge V \in \mathcal{G} \\ 0 & \text{otherwise} \end{cases} \quad (3.11)$$

3.1.5 Group-Level Encoding and Prediction

The same Community-Centric Encoder from Step 1 is reused on this new group graph $\hat{G}$ with shared weights:

$$\hat{X}^{(3)} = \text{CommunityCentricEncoder}(\hat{X}, \hat{E}) \quad (3.12)$$

$$p_{\text{group}} = \sigma(W_c \hat{X}^{(3)}) \quad (3.13)$$

where $\hat{X}^{(3)}$ is the resulting group-level embedding, and p_{group} is the final illicit probability prediction obtained by performing a classification step on these embeddings. Sharing weights enforces consistency between the node and group levels while keeping the parameter count low.

3.1.6 Multi-Task Loss

Unlike standard GNNs that optimize a single objective, GAGNN relies on three supervised signals. The total loss $\mathcal{L}$ is a weighted sum of the node-level, transaction-level, and group-level losses:

$$\mathcal{L} = c_1 \mathcal{L}_{\text{group}} + c_2 \mathcal{L}_{\text{node}} + c_3 \mathcal{L}_{\text{trans}} \quad (3.14)$$

where $c_1, c_2, c_3 \in \mathbb{R}^+$ are hyperparameters and $\mathcal{L}_{\text{group}}$, $\mathcal{L}_{\text{node}}$ and $\mathcal{L}_{\text{trans}}$ are the group-level, node-level, and transaction-level losses, respectively. Since nodes and groups are strictly classified as illicit or legitimate, their losses are always weighted Binary Cross-Entropy (BCE) functions:

$$\mathcal{L}_{\text{node}} = \text{BCE}_w(p_{\text{node}}, y_{\text{node}}) \quad (3.15)$$

$$\mathcal{L}_{\text{group}} = \text{BCE}_w(p_{\text{group}}, y_{\text{group}}) \quad (3.16)$$

For the transaction classification, $\mathcal{L}_{\text{trans}}$ is a BCE loss for binary tasks, and a Cross-Entropy (CE) loss for multiclass classification.

$$\mathcal{L}_{\text{trans}} = \begin{cases} \text{BCE}_w(p_{\text{trans}}, y_{\text{trans}}) & \text{if binary task} \\ \text{CE}_w(p_{\text{trans}}, y_{\text{trans}}) & \text{if multiclass task} \end{cases} \quad (3.17)$$

The positive-class weight w is tuned to penalize false negatives, in the finance sector detecting most of the frauds is more important than maximizing the accuracy. By optimizing this joint loss, the model learns to identify money laundering at the individual, transaction, and community levels simultaneously.

3.2 DATASET DESCRIPTION

The IBM AMLSim dataset comes in 6 different versions.

- HI-Small
- HI-Medium
- HI-Large

	Small		Medium		Large	
Statistic	HI	LI	HI	LI	HI	LI
# of Days Spanned	10	10	16	16	97	97
# of Bank Accounts	515K	705K	2077K	2028K	2116K	2064K
# of Transactions	5M	7M	32M	31M	180M	176M
# of Laundering Transactions	3.6K	4.0K	35K	16K	223K	100K
Laundering Rate (1 per N Trans)	981	1942	905	1948	807	1750

Figure 4: IBM AMLSim datasets statistics [2]

- LO-Small
- LO-Medium
- LO-Large

Where HI (High) and LO (Low) indicate the frequency of illicit transactions and the suffixes Small, Medium and Large indicate the size of the dataset. Figure 4 shows the differences between the 6 datasets.

In this work we will use the HI-Small version. Bigger datasets would of course guarantee a better generalization and accuracy but, for hardware and time constraint, it would be impossible to train the models on such graphs. That begin said, we believe that the obtained results are good enough to understand how the models would perform in a real world scenario.

The HI-Small dataset is composed of 3 different files:

- `HI-Small_account.csv`, contains 518,581 accounts together with their features:
 - *Entity ID*: The ID of the account;
 - *Entity Name*: The name of the account;
 - *Bank Name*: The name of the bank holding the account;
 - *Bank ID*: The ID of the bank holding the account;
 - *Account Number*: The number of the account;

Figure 5 shows some examples of account records.

- `HI-Small_Trans.csv`, contains transactions features and binary labels:
 - *Timestamp*: The timestamp of the transaction;
 - *From Bank*: The ID of the sending bank;

	Bank Name	Bank ID	Account Number	Entity ID	Entity Name
0	Portugal Bank #4507	331579	80B779D80	80062E240	Sole Proprietorship #50438
1	Canada Bank #27	210	809D86900	800C998A0	Corporation #33520
2	UK Bank #33	21884	80812BE00	800C47F50	Partnership #35397
3	Germany Bank #4815	32742	81047F300	80096F0B0	Corporation #48813
4	National Bank of Harrisburg	127390	80BD8CF00	800FB8760	Corporation #889

Figure 5: IBM AMLSim account records

- *From Account*: The ID of the sending account;
- *To Bank*: The ID of the receiving bank;
- *To Account*: The ID of the receiving account;
- *Amount Received*: The amount of money received by the recipient;
- *Receiving Currency*: The currency of the received amount (USD, EUR, etc.);
- *Amount Paid*: The amount of money paid by the sender;
- *Payment Currency*: The currency of the paid amount;
- *Payment Format*: The format/method of the payment;
- *Is Laundering*: Binary label indicating whether the transaction is illicit;
- *src*: Source node identifier;
- *dst*: Destination node identifier;

Figure 6 shows some examples of transaction records.

- *HI-Small_Patterns.txt*, lists the edges that are involved in ML activities together with the type of pattern they are involved in (multiclass labels). Examples of the 8 different types of ML patterns in IBM AMLSim are shown in Figure 7.

3.2.1 Features analysis

In this section we provide a detailed analysis of the edges and nodes features distributions. Before diving into that we first take a look at the distribution of the labels in the dataset. There are 5,177 illicit transactions over a total of 5,078,345 Edges. Figure 8, provides the distribution of the

	Timestamp	From Bank	From Account	To Bank	To Account	Amount Received	Receiving Currency	Amount Paid	Payment Currency	Payment Format	Is Laundering	src	dst
0	2022-09-01 00:20:00	010	8000FBD30	010	8000FBD30	3697.34	US Dollar	3697.34	US Dollar	Reinvestment	0	010_8000FBD30	010_8000FBD30
1	2022-09-01 00:20:00	03208	8000F4580	001	8000F5340	0.01	US Dollar	0.01	US Dollar	Cheque	0	03208_8000F4580	001_8000F5340
2	2022-09-01 00:00:00	03209	8000F4670	03209	8000F4670	14675.57	US Dollar	14675.57	US Dollar	Reinvestment	0	03209_8000F4670	03209_8000F4670
3	2022-09-01 00:02:00	012	8000F5030	012	8000F5030	2806.97	US Dollar	2806.97	US Dollar	Reinvestment	0	012_8000F5030	012_8000F5030
4	2022-09-01 00:06:00	010	8000F5200	010	8000F5200	36682.97	US Dollar	36682.97	US Dollar	Reinvestment	0	010_8000F5200	010_8000F5200

Figure 6: IBM AMLSim transaction records

multiclass label (type of pattern) over the fraudulent edges. It's important to notice that this information is provided only for 3,149 out of the 5,177 illicit transactions. This issue will be addressed in Section 3.3.1.

We now proceed by analyzing the edge features. First, we notice that *Amount Received* and *Amount Paid* are almost identical. From Figure 9 we can see that fraudulent transactions have the tendency to be bigger than legit ones.

Continuing our analysis on edge features, we notice that the most common currency is USD for both fraudulent and legit transactions; most of the currencies have a similar proportion distribution from both fraudulent and legit transactions. From Figure 10 we can see that there are two exceptions: Shekel, which is more prevalent in legit transactions, and Saudi Riyal, which is significantly more prevalent in fraudulent transactions. It's important to notice that the graph shows the proportion of each currency with respect to the total number of transactions in each class; Since there are way more legit than fraudulent transactions, having similar distributions means that, for every type of currency, legit transactions still represent the vast majority.

Considering now the payment formats, by looking at Figure 11 we notice that ACH transactions are more frequently used for fraudulent transactions compared to other methods.

The last edge feature is *Timestamp*. Since working with timestamps can be tricky, we decided to extract to categorical features representing the hour of the day and the day of the week (Figures 12). Normally we would have also considered the week of the year and the year itself but, since IBM AMLSim Small only covers the span of a week, it would be irrelevant. That being said, it's important to mention that directly working with timestamps, albeit tricky, could lead to a more precise detection [Chapter 5.1].

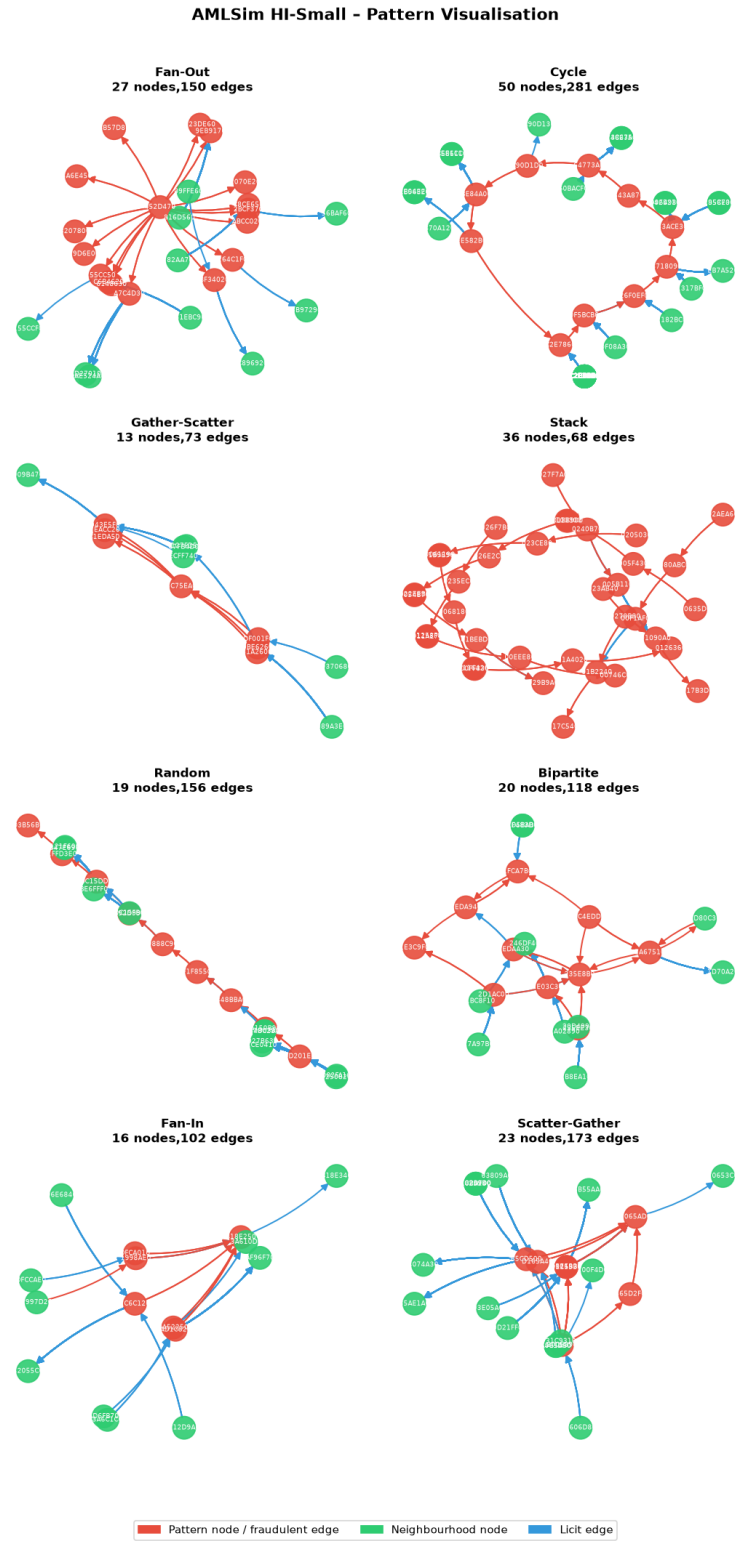


Figure 7: IBM AMLSim patterns examples

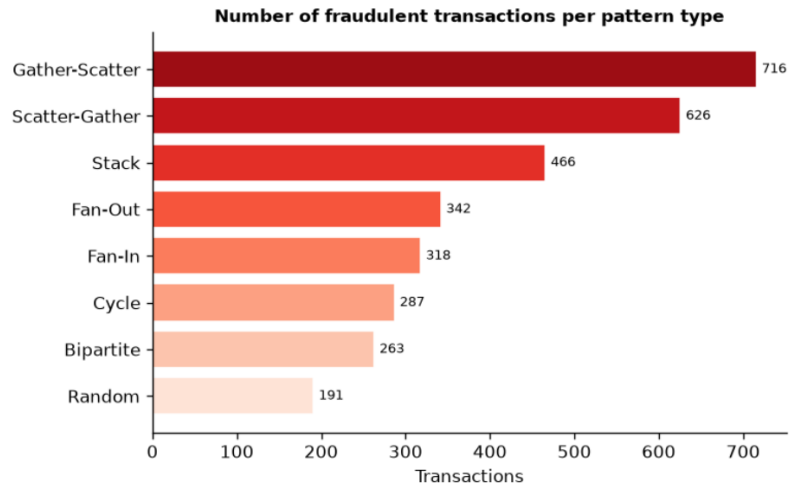


Figure 8: IBM AMLSim patterns distribution over fraudulent edges

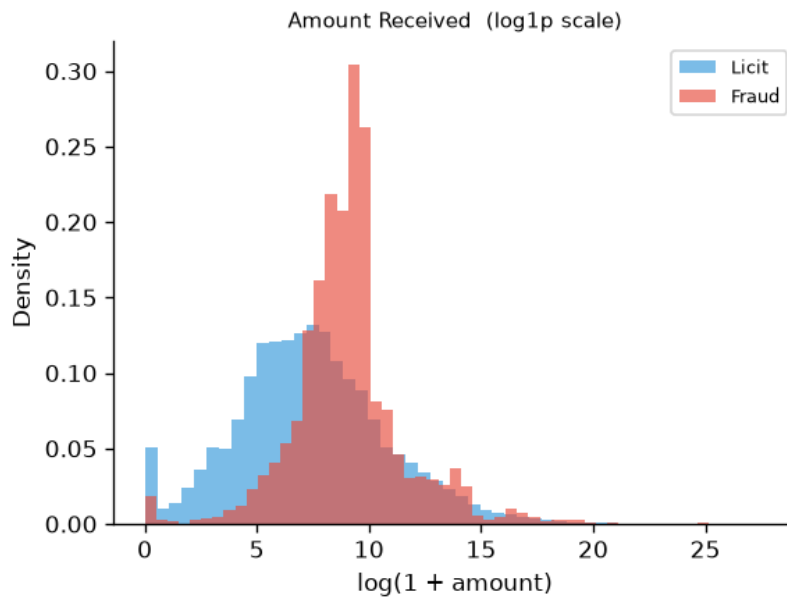


Figure 9: IBM AMLSim amounts distribution

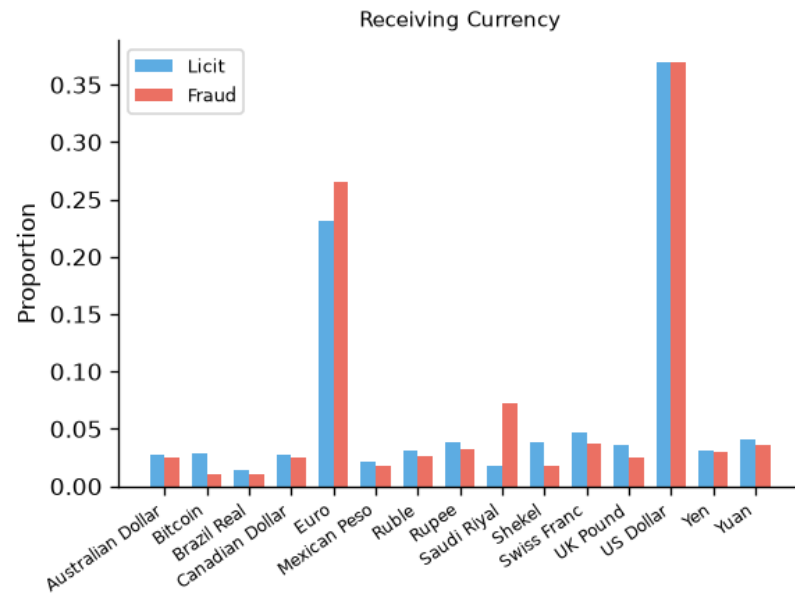


Figure 10: IBM AMLSim currencies distribution

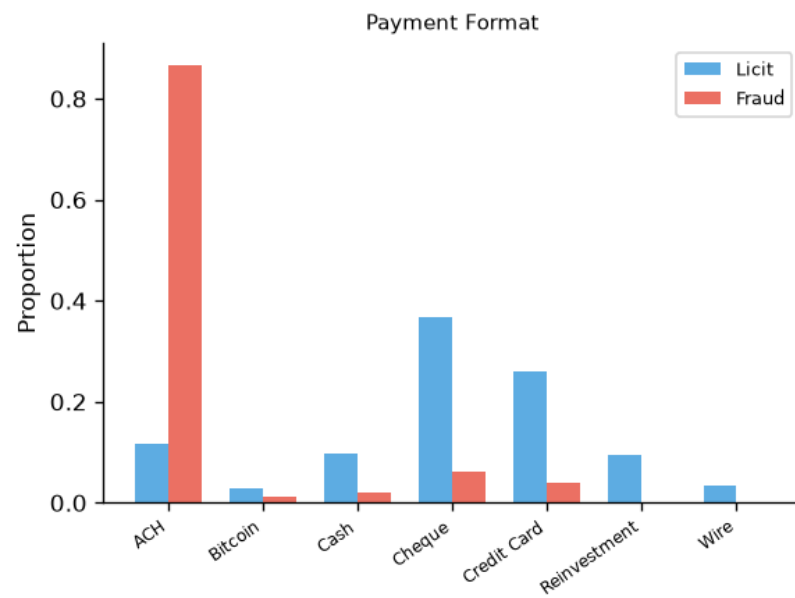


Figure 11: IBM AMLSim payment formats distribution

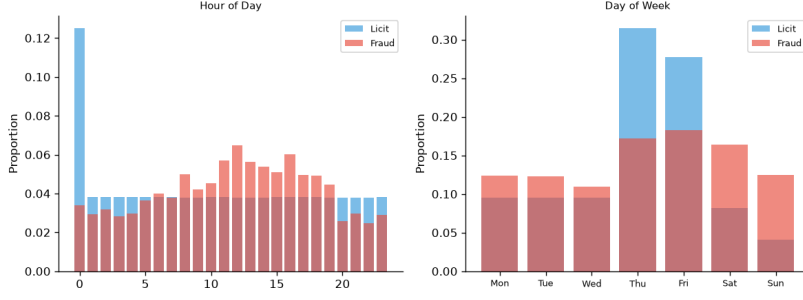


Figure 12: IBM AMLSim hour and day of the week distributions

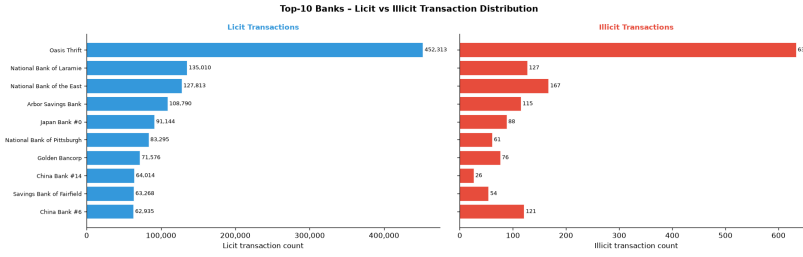


Figure 13: IBM AMLSim banks distribution

Finally we take a look at the Node features. The only interesting one is *Bank Name*, Figure 13 shows the distribution of licit and illicit nodes over the 10 biggest banks.

3.2.2 Building the Financial Graph

Since the IBM AMLSim Dataset account (Node) features are very limited, we will leverage the transaction (Edge) features to derive the Node's ones.

Let $\mathcal{N}(i)$ be the set of neighbors of account i , e_{ij} the features vector of a transaction from account i to account j , y_{ij} the label of that transaction. The feature vector x_i for the account i and his ground truth label $\hat{y}_i$ are defined as follows.

$$x_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} e_{ij} \quad (3.18)$$

$$\hat{y}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} y_{ij} \quad (3.19)$$

It's important to mention that in case of qualitative edge features or multiclass edge labels, the mean will be computed on the one-hot encoded vectors.

3.3 EXPERIMENTAL SETTINGS

3.3.1 Data Preparation

In regard to enhance the performance of the models we perform some preprocessing steps on the dataset. First of all, categorical features need to be converted in a numerical format; we opted for One-Hot Encoding, which works particularly well for our architecture since, when we average the features to create the nodes and groups representations, the one-hot vectors translate into proportions of the different categories providing like that meaningful information.

Secondly, we elaborated the timestamp by extrapolating from it the hour of the day, using cyclic encoding, and the day of the week, using a one-hot encoder. The cyclic representation is fundamental to preserve the daily periodicity of transactions.

We then normalized the financial amounts. Since money laundering datasets present a high number of outliers standard normalization techniques often tend to move the value of the legitimate transactions close to zero. For this reason a Robust Scaler was preferred to the standard one. It normalizes features using the median and the interquartile range (IQR), according to the following formula:

$$x_{\text{scaled}} = \frac{x - Q_2(x)}{Q_3(x) - Q_1(x)} \quad (3.20)$$

where $Q_2(x)$ is the median, and $Q_1(x)$ and $Q_3(x)$ are the 25th and 75th percentiles of the feature values. This prevents extreme values from distorting the distribution of the data.

Finally, we extrapolated the multiclass labels from `HI-Small_Patterns.txt` obtaining the categorical feature. As before, we then apply one-hot encoding to allow the model to understand it.

3.3.2 Train, Patience, Validation and Test Split

Anti Money Laundering algorithms are usually performed on the new incoming transactions to detect fraudulent activity. This means that the models must learn from the past to predict the future. For this reason,

splitting the dataset randomly would inevitably cause future information to leak into the past. To avoid that we decided to split our dataset by using 3 temporal thresholds t_1, t_2, t_3 that define the training, patience, validation and test sets as follows.

$$\mathcal{D}_{\text{train}} = \{d \in \mathcal{D} | t(d) \leq t_1\} \quad (3.21)$$

$$\mathcal{D}_{\text{patience}} = \{d \in \mathcal{D} | t_1 < t(d) \leq t_2\} \quad (3.22)$$

$$\mathcal{D}_{\text{val}} = \{d \in \mathcal{D} | t_2 < t(d) \leq t_3\} \quad (3.23)$$

$$\mathcal{D}_{\text{test}} = \{d \in \mathcal{D} | t(d) > t_3\} \quad (3.24)$$

where $t(d)$ is the timestamp of the transaction d , and t_1, t_2 and t_3 are specially picked so that $|\mathcal{D}_{\text{train}}| \approx 0.7|\mathcal{D}|$, $|\mathcal{D}_{\text{patience}}| \approx 0.075|\mathcal{D}|$, $|\mathcal{D}_{\text{val}}| \approx 0.075|\mathcal{D}|$ and $|\mathcal{D}_{\text{test}}| \approx 0.15|\mathcal{D}|$. The patience set is exclusively used to trigger early stopping during both model selection and training. The validation set is instead kept strictly for evaluating and comparing models during the selection phase, preventing the evaluation metric from being biased by the early stopping mechanism.

During the training phase, since we no longer need 4 different subsets, the train set becomes $\mathcal{D}_{\text{train}} \leftarrow \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{patience}}$, the validation will be used for the early stopping $\mathcal{D}_{\text{patience}} \leftarrow \mathcal{D}_{\text{val}}$ and the test set remains untouched to provide an unbiased evaluation of the model performance. This ensures that the model always trains on a contiguous block of time and prevents data from the future to leak into the past.

It's important to mention that, in order to prevent data leakage, the Robust Scaler was fitted exclusively on the transactions belonging to the training split, and the resulting parameters (Q_1, Q_2, Q_3) were then used to independently transform the patience, validation and test sets.

Finally, in order to address the class imbalance, we decided to apply downsampling on the training set. For the binary task, the majority class (legitimate accounts) is downsampled to match the number of illicit accounts, achieving a 1:1 ratio. For the multiclass task, the legitimate class is downsampled to match the total sum of all illicit instances across the different illicit patterns. This achieves a 1:1 ratio between legitimate and overall illicit samples, while preserving the natural distribution and relative frequencies among the specific illicit classes. The validations and test sets remain untouched.

Test set problem

By analysing the test set we noticed a flaw in the IBM AMLSim dataset. The amount of ML transactions suddenly increases in the last rows,

Interval	Percentage of ML transactions
Last 100,000 rows	0.9%
Last 10,000 rows	6.7%
Last 5,000 rows	13.2%
Last 1,000 rows	59.0%

Table 1: ML transactions distribution in the last rows of the dataset

corresponding to the newer transactions, of the dataset. Because of that we initially ended up with a test set that didn't really represented the real world scenario. The distribution of the ML attempts in the last rows is displayed in Table 1. For this reason we decided to get rid of the last 5000 rows by re-defining the test set as:

$$\mathcal{D}_{\text{test}} = \{d \in \mathcal{D} | t_3 < t(d) \leq t_4\} \quad (3.25)$$

where t_4 is specially chosen to achieve the objective.

3.4 MODEL SELECTION AND TRAINING SETTINGS

In this section we describe and explain our choices for the training settings and the models' hyperparameters. We first discuss the general settings shared amongst all the experiments, then we deep dive into the architectural choices and hyperparameter tuning for each class of models we evaluated.

3.4.1 *Optimizer*

As suggested by Cheng et al. [5], we decided to adopt the Adam optimizer [12] as the optimization algorithm for all our models. The original implementation suggested a learning rate of 0.001 and a batch size of 128. With our dataset these settings led to a very unstable convergence, therefore we decided to lower the learning rate to $\eta = 0.0001$ which, together with gradient clipping at a maximum norm of 5.0, ensured smoother and more reliable training dynamics. Furthermore we also changed the batch size to 256 to better exploit the GPU usage.

3.4.2 Subgraph Sampling

Due to the prohibitive dimension of the financial graph, fitting the models directly to the entirety of it is not a feasible option in both terms of memory and time.

To address this, we employed a mini-batch subgraph sampling strategy using NeighborLoader from PyTorch Geometric [10]. The loader operates by sampling a fixed number of neighbors per hop. This is governed by the `num_neighbors = [a, b, c, ...]` parameter. Meaning that for each target node in the batch, the loader samples `a` of its immediate neighbors; for each of those `a` sampled neighbors, it further samples `b` of their neighbors and so on. In our experiments, we set `num_neighbors = [15, 10, 10]` across $L = 2$ or $L = 3$ hops depending on the model architecture. This preserves the local graph topology while keeping the computational cost manageable.

This sampling strategy is only used during training. For validation and testing, we set `num_neighbors = [-1, -1, -1]`, meaning that all neighbors are included. This guarantees an unbiased evaluation of the model's performance on the realistic, imbalanced data.

3.4.3 Early Stopping

To prevent overfitting we employed an early stopping mechanism that monitors the loss on the patience set $\mathcal{D}_{\text{patience}}$. Training is stopped when no improvement is observed for a specified number of consecutive epochs (the *patience*). During the model selection phases, we set a maximum of 3000 epochs with a patience value of $p_{\text{selection}} = 100$; during the final training phase, we set a maximum of 6000 epochs with a patience value of $p_{\text{training}} = 150$, to allow the model to fully converge on the selected hyperparameters. These settings are shared across all the experiments.

3.4.4 Hyperparameters

We begin by listing the hyperparameters that will need to be tuned across all the different models.

LOSS FUNCTION HYPERPARAMETERS

- c_1 : Weight for the group-level loss $\mathcal{L}_{\text{group}}$.
- c_2 : Weight for the node-level loss $\mathcal{L}_{\text{node}}$.

- c_3 : Weight for the transaction-level loss $\mathcal{L}_{\text{trans}}$.
- w (laundry_weight): Positive-class weight used in the BCE loss.

GAGNN HYPERPARAMETERS

- β : Trade-off parameter in the Extended Markov Random Field (eMRF) layer that balances the influence of topological similarity ε and attribute similarity R_ζ (Section 3.1.2).
- `mlp_hidden_dim`: Dimension of the hidden layers in the MLP_{node} , used to map the refined embeddings $X^{(3)}$ to node-level classification logits (Section 3.1.3).
- `nn_t_hidden_dim`: Dimension of the hidden layers in the MLP_{edge} , used to compute the transaction probability p_{trans} (Section 3.1.4).
- `out_dim`: Dimension d_{out} of the enriched embeddings $X^{(2)}$ (Section 3.1.1).

GAT HYPERPARAMETERS

- `hidden_dim`: Dimension of the hidden layers in the message passing steps.
- `L (num_layers)`: Number of message passing layers, determining the multi-hop receptive field of the GNN.
- `H (heads)`: Number of attention heads.

GIN HYPERPARAMETERS

- `hidden_dim`: Dimension of the hidden layers in the message passing steps.
- `L (num_layers)`: Number of message passing layers, determining the multi-hop receptive field of the GNN.
- `gin_mlp_hidden`: Dimension of the hidden layers of the MLP used inside the GIN UPDATE function (Section 2.2.3).

Due to the very high number of hyperparameters that need to be tuned, we decided to split the model selection process into two distinct phases. The first one concentrates on finding the best values for the parameters involved in the loss function by keeping the architectural hyperparameters

Parameter	Value	Rationale
L	2	original paper value
H	5	original paper value
β	0.44	original paper value
hidden_dim	64	original paper value
nn_t_hidden_dim	128	original paper value
out_dim	128	assumed as hidden_dim *2
mlp_hidden_dim	128	assumed as hidden_dim *2

Table 2: Fixed architectural hyperparameters for Phase 1

fixed to reference values suggested by Cheng et al. [5]. The second phase instead focuses on tuning the architectural hyperparameters (hidden dimension, number of layers, number of attention heads, etc.) keeping the loss function weights fixed to the values found in the first one.

Loss Function Hyperparameters

In this phase we search for the optimal values of the parameters c_1, c_2, c_3 and w (used only in the binary task), involved in the loss functions.

To save computational resources, the grid search for the loss weights is performed exclusively on the GAGNN model with the GAT backbone, assuming that the optimal weights found for GAT apply directly to the GIN backbone as well for their respective tasks. During this phase, all architectural hyperparameters were kept fixed to reference values suggested by Cheng et al. [5]; when not explicitly stated we opted for feasible and reasonable values according to the literature. The fixed architectural hyperparameters are reported in Table 2.

A critical design choice in this phase concerns the evaluation metric. Since the validation loss $\mathcal{L}_{\text{val}}$ is directly affected by the parameters c_1, c_2, c_3 and w , using it as a selection criterion would be a mistake. We therefore leveraged the macro F1-score on the validation set as the model selection criterion. Validation loss still served to trigger the early stopping.

The hyperparameters grids for binary and multiclass tasks are reported in Table 3 and Table 4 respectively.

For the multiclass task the weight w is replaced by automatically computed per-class weights using `sklearn` with `class_weight='balanced'`.

Parameter	Searched values	Rationale
c_1	{0.75}	Fixed
c_2	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75
c_3	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75
w (laundry weight)	{1.0, 2.0}	to penalize false negatives

Table 3: Grid search for binary loss function hyperparameters

Parameter	Searched values	Rationale
c_1	{0.75}	Fixed
c_2	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75
c_3	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75

Table 4: Grid search for multiclass loss function hyperparameters

We denote the optimal values produced by the grid searches as $c_1^{*binary}$, $c_2^{*binary}$, $c_3^{*binary}$, w^* and c_1^{*multi} , c_2^{*multi} , c_3^{*multi} .

Architecture Hyperparameters

The second phase consists in finding the optimal values for the architectural hyperparameters, keeping the loss function weights fixed to the values found in the first phase. This allows to use the *validation loss* $\mathcal{L}_{val}$ instead of the validation F1-score as selection criterion; this is a better choice in this case since the F1-score metric is defined by strict decisions boundaries and therefore is not continuous and more sensitive to small fluctuations.

We will perform four different model selections, one for each experiment: GAGNN with GAT backbone for the binary task, GAGNN with GIN backbone for the binary task, GAGNN with GAT backbone for the multiclass task and GAGNN with GIN backbone for the multiclass task. Two hyperparameters grid are used, since they are shared between binary and multiclass tasks with the same backbone GNN. As possible values we chose options around those of the reference paper; the hyperparameters grids for GAT and GIN backbones are reported in Table 5 and Table 6 respectively.

Trying more parameters, expecially for the number of layers, number of heads, β and coefficients of the loss function could lead to much better results. However, due to the limitations on the computational resources,

Parameter	Searched values
H	5
L	[2,3]
β	0.44
hidden_dim	[64, 128]
nn_t_hidden_dim	[128, 256]
out_dim	[128, 256]
mlp_hidden_dim	[128, 256]

Table 5: Grid search for GAGNN with GAT

Parameter	Searched values
L	[2,3]
β	0.44
hidden_dim	[64, 128]
nn_t_hidden_dim	[128, 256]
out_dim	[128, 256]
mlp_hidden_dim	[128, 256]
gin_mlp_hidden	[128, 256]

Table 6: Grid search for GAGNN with GIN

we had to limit the grid search to a reasonable number of combinations. We will discuss this further in Section 4.3.

Baseline Models Hyperparameters

For the baseline models (GIN and GAT) we used as hyperparameters the same values obtained for the internal GNN encoder of their respective GAGNN counterpart. This choice stems from the need to understand how much the GAGNN architecture enhances the GNNs capacity of detecting fraudulent transactions, without the confounding factor of different hyperparameter settings.

RESULTS

In this chapter we present and discuss the results obtained by the previously described models on the dataset IBM AMLSim small [2]. We will first present the results of model selection (Sections 4.1, 4.1.2), to then proceed by analysing the results of the GAGNN models (Section 4.2) and comparing them to the baseline models to evaluate the effectiveness of our proposed method.

4.1 HYPERPARAMETER SELECTION RESULTS

As described in Section 3.4.4 and Section 3.4.4, the model selection process is splitted into two phases to mitigate the computational cost of the grid search. As we will see, this will come at the cost of a sub-optimal model but due to the limitations of our hardware this is still the best option.

4.1.1 *Loss Function Hyperparameters*

We began by performing the grid search on the loss function parameters for both binary and multiclass tasks using the GAGNN with GAT backbone. Table 7 and Table 8 show the optimal hyperparameters found by this process by optimizing the macro F1-score on the validation set.

As we can see in both binary and multiclass tasks the weight associated with the edge loss c_3 is the highest value. Making the edge loss the more significant to the model performance. Especially in the multiclass task, this is expected since the edge loss is the one that predicts the actual target variable, meaning the patterns, whilst the node loss and the group loss are only based on binary classification.

Another interesting observation is that, in the binary task, the weight associated with the node loss c_2 is lower than the one for the group loss c_1 . This suggests that the idea of trying to force the model to predict money

Parameter	Optimal Value
c_1	0.75
c_2	0.5
c_3	1.0
w (laundry weight)	2

Table 7: Optimal loss function hyperparameters for the binary task

laundering at a community level might be a good one. The aggregation into communities allows the model to indirectly expand his receptive field without adding more layers to the GNN architecture, this should mitigate the cances of incurring into oversmoothing.

In the multiclass this is no longer true, my hipotesis is that while the community prediction helps us detecting whether a community is involved in money laundering or not, it does not help us to detect the exact patterns; once the fraudulent nodes are merged into communities we can no longer understand what type of pattern generated that community and therefore the model loses the ability to predict it.

Finally, note how in the binary task the best parameters for c_2 , c_3 and w lay at the border of the hyperparameter search space suggesting that future works should further investigate these values.

Parameter	Optimal Value
c_1	0.75
c_2	0.75
c_3	0.9

Table 8: Optimal loss function hyperparameters for the multiclass task

4.1.2 Architectural Hyperparameters

In the second phase of model selection, we runned 4 separate grid searches for the GAGNN-GAT (Binary), GAGNN-GAT (Multiclass), GAGNN-GIN (Binary) and GAGNN-GIN (Multiclass) architectures to find the optimal hyperparameters for each of them. The results are displayed in Table 9, 10, 11 and 12 respectively.

The first thing to notice is that, in case of GAGNN-GAT (Binary), which is the same task tackled by the paper [5], although on a different dataset,

Parameter	Optimal Value
H	5
L	2
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	128
mlp_hidden_dim	128

Table 9: Optimal architectural hyperparameters for GAGNN with GAT (Binary task)

we found different optimal parameters. In particular they mention as best parameters `hidden_dim = 64` and `nn_t_hidden_dim = 128`, which differ from our best parameters `hidden_dim = 128` and `nn_t_hidden_dim = 256`. The number of layers `L` is the same as the one found in [5].

For the number of attention heads and the value of β we trusted the paper and used the same hyperparameters due to the high computational cost that would have come with their tuning. That being said, the differences in `hidden_dim` and `nn_t_hidden_dim` suggests that tuning all the architectural hyperparameters could lead to big improvements in the model performance.

Another interesting observation is that for both GAGNN-GIN and GAGNN-GAT the optimal parameters for the multiclass task suggest the need of a bigger model compared to the binary task. This is expected since usually more complex tasks require more complex models. As we will see Section 4.2, those bigger models led to overfitting suggesting, in case of multiclass classification, the need of a bigger dataset to successfully complete the task. Due to our hardware limitations we couldn't afford to do that, but future works should consider using the IBM AMLSim Large Dataset to overcome this issue.

Finally we notice how in GAGNN-GIN (Multiclass) the optimal number of layers is 3 which is different from the 2 of the binary case. This results together with the others displayed in table 12 are to be considered with caution; as we will see GAGNN-GIN for the multiclass task didn't converge during the training and therefore the results are not reliable.

Parameter	Optimal Value
H	5
L	2
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	256
mlp_hidden_dim	256

Table 10: Optimal architectural hyperparameters for GAGNN with GAT (Multi-class task)

Parameter	Optimal Value
L	2
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	128
mlp_hidden_dim	128
gin_mlp_hidden	256

Table 11: Optimal architectural hyperparameters for GAGNN with GIN (Binary task)

Parameter	Optimal Value
L	3
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	128
mlp_hidden_dim	128
gin_mlp_hidden	256

Table 12: Optimal architectural hyperparameters for GAGNN with GIN (Multi-class task)

4.2 MODELS RESULTS

In this section we present and discuss the results obtained by training GAGNN with GAT and GIN backbones on both binary and multiclass classification tasks and we compare them with the results obtained by training standard GNNs on the same tasks.

As expected the binary task was easier to solve. Due to our hardware limitations we cannot afford to use a larger dataset that would allow the training of bigger models and with that better performances on the multiclass task. The question that we will try to answer is whether the GAGNN architecture could be useful also for multiclass detection or if it only performs well on the binary task.

4.2.1 Evaluation Metrics

Let TP be the number of True Positives, TN the number of True Negatives, FP the number of False Positives, and FN the number of False Negatives. In regard to evaluate the performance of our models, we will leverage the following classification metrics.

- **Accuracy:** The ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.1)$$

- **Precision:** The ratio of correctly predicted positive observations to the total predicted positive observations.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4.2)$$

- **Recall:** The ratio of correctly predicted positive observations to all observations in actual class.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4.3)$$

- **F1-Score:** The harmonic mean of Precision and Recall, which provides a balance between the two.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.4)$$

- **ROC-AUC:** The Area Under the Receiver Operating Characteristic (ROC) Curve. The ROC curve plots the True Positive Rate (TPR = Recall) against the False Positive Rate ($FPR = \frac{FP}{FP+TN}$) across all possible classification thresholds. While TPR measures the proportion of actual illicit transactions correctly identified, FPR measures the proportion of legitimate transactions incorrectly flagged as illicit. The AUC (Area Under the Curve) provides an aggregate measure of performance independent of any specific threshold, representing the probability that the model ranks a randomly chosen positive example higher than a randomly chosen negative one. A value of 0.5 indicates a model that performs no better than random guessing, while a value of 1.0 represents a perfect classifier. The ROC-AUC is especially informative in tasks with highly imbalanced classes, such as Anti-Money Laundering, where standard accuracy can be misleading due to the overwhelming majority of legitimate transactions.

In the multiclass setting, these metrics are computed using a macro or weighted average by treating each class as a separate binary classification problem.

4.2.2 General considerations

We begin by providing some considerations regarding training dynamics that are common to all the models.

As we can see in Figures 14, 26, 17, 29, 20, 32, 23 and 35, the training loss and the patience loss, albeit less for the latter, tend to be a bit noisy. This is mainly due to the fact that the losses are computed on mini batches of 256 nodes instead of on the entire dataset. Furthermore, while is true that for the validation loss we consider all the neighborhood of the sampled nodes ($num_neighbors = [-1,-1,-1]$); for the training mini-batches, in regard to reduce the computational cost, we only considered 15 of the adjacent neighbors, 10 neighbors for each of those 15 and, in case of a 3 layer GNN, another 10 neighbors for each of those 10×15 ($num_neighbors = [15, 10, 10]$). This introduces further noise in the loss computation and it's probably the reason why in some experiments the training loss is not as smooth as we expect it to be.

Another interesting thing is how the GIN models tend to converge faster than the GAT ones; this is probably due to the five heads ($H = 5$) used in the GAT model which introduces more parameters to train and with that

a larger search space for the optimization algorithm to explore. Probably for the same reasons, baseline models converge faster than GAGNN models, which again introduce more parameters to train.

An important thing to mention about the graph plots is that they show the training loss and the patience loss only up to the point in which the early stopping policy was triggered. This is clear when we look at the training history for the GAGNN-GIN Multiclass task (Figure 29), where the model fails to converge and the best value reached in the 12th epoch and doesn't improve in the following 150 epochs.

Finally, it is important to reflect on the theoretical strengths of the different architectures employed and how they map to specific money laundering patterns. Group-aware models like GAGNN are explicitly designed to capture organized, gang-level behaviors. Theoretically, they should excel at identifying structures as fraudulent (though not necessarily telling them apart) such as gather-scatter, fan-in, fan-out, bipartite, and stack patterns. This is because it should be fairly easy for the model to merge the fraudulent edges in those structures into a single community node. It is important to clarify that while this merging mechanism is extremely beneficial in the binary classification task for distinguishing illicit clusters from normal traffic, it may actually hinder performance in the multiclass task: grouping nodes into supernodes might help detect that a transaction is fraudulent, but it obfuscates the specific topological structure needed to accurately classify the exact pattern. Nonetheless, the multiclass results partially support this theoretical advantage, as GAGNN-GAT proved competitive on the gather-scatter pattern. However, this good performance might also depend on the fact that gather-scatter is the best-represented class in the dataset, with 716 labeled edges (see Figure 8).

For random and cycle patterns, the situation is different. Since these patterns can involve an arbitrary number of nodes, and our GNNs are restricted to 2 or 3 layers, the limited receptive field makes it difficult to group all of the involved nodes into a single community. Nevertheless, GAGNN should still improve the binary detection of those patterns compared to a standard GNN by allowing different nodes to merge, thus indirectly expanding the receptive field of the nodes. In our experiments, however, the model struggled significantly with the cycle class. This is likely due to the extreme scarcity of this pattern, as there are only 287 edges labeled as cycle in the entire dataset (Figure 8), which prevented the complex group-aggregation layers from learning the pattern without overfitting.

Furthermore, we must mention that the bipartite class is not correctly represented in the IBM AMLSim dataset, as it is not implemented as described in the original AMLWorld paper. Comparing the theoretical representation of the bipartite class shown in Figure 1 with the actual dataset implementation provided in Figure 7, it is clear that the topological structure differs significantly. This discrepancy is likely the main reason why the bipartite class is not well recognized in the multiclass task and could also lead to a lower performance in the binary classification task.

Regarding the backbone architectures, the core strength of GAT lies in its ability to compute hidden representations by attending over neighbors with different, learnable weights. This makes it highly effective for hub-and-spoke structures such as fan-in and fan-out patterns. The attention mechanism allows the model to assign disproportionately high weights to the anomalous edges connecting to the central hub account, effectively filtering out the noise from legitimate transactions. Lastly, GIN is theoretically proven to be as discriminative as the Weisfeiler-Lehman graph isomorphism test, making it exceptionally powerful at distinguishing exact structural graph topologies. Since it relies on sum-aggregation to capture the exact multiset of neighborhood features, GIN is best suited for rigid, structurally distinct patterns like stack or specific chained topologies, where the exact structural signature is the defining characteristic of the laundering behavior.

4.2.3 Binary Classification task

When analyzing the results for the binary classification task, it is crucial to establish the priority of the metrics involved. In the context of Anti-Money Laundering (AML), the primary objective is to detect as many illicit transactions as possible. Therefore, achieving a high Recall on the fraudulent class (labeled as *Laundering*) is far more important than maximizing overall accuracy or precision. Failing to flag a money laundering attempt as suspicious (a False Negative) could lead to severe legal and financial repercussions for a financial institution, which vastly outweigh the operational cost of investigating a legitimate account that was falsely flagged (a False Positive).

Bearing this in mind, the GAGNN-GIN model yielded the most compelling results from a strict risk-aversion standpoint. It achieved an outstanding recall of 0.86 on the Laundering class, significantly outperforming the standard GIN baseline (0.71). However, this high sensitivity came

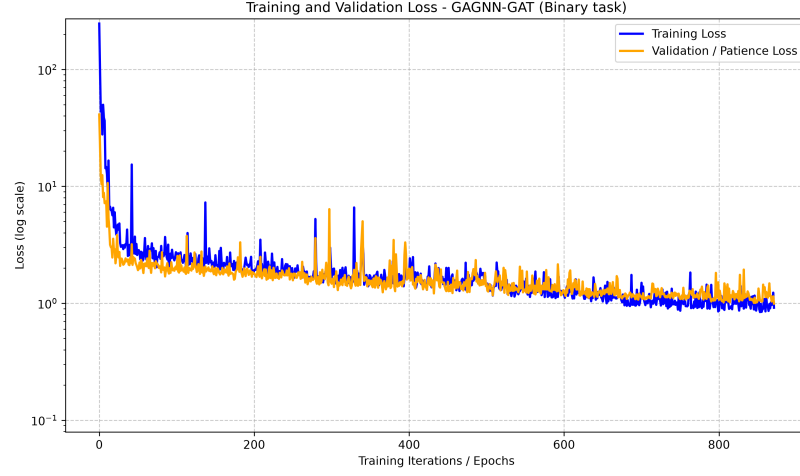


Figure 14: Training and patience loss for GAGNN-GAT (Binary task)

at the cost of precision (0.54), leading to a higher number of false positives which dragged its overall accuracy down to 0.8778.

If a more balanced approach is desired, the GAGNN-GAT model proved to be the best overall performer. It achieved the highest Accuracy (0.9268), ROC-AUC (0.9065), and F1-Score (0.75) on the minority class among all the tested models. Crucially, it maintained a strong recall of 0.76 while keeping precision relatively high (0.73), proving that the GAGNN architecture successfully improved upon the standard GAT baseline across every single metric.

4.2.4 Multiclass Classification task

Unlike the binary setting, the multiclass classification task proved to be exceedingly difficult for the GAGNN architecture. The best performance was surprisingly achieved by the standard GIN baseline model, which reached an accuracy of 0.4526 and a ROC-AUC of 0.7408. In contrast, the GAGNN models struggled significantly: the GAGNN-GAT achieved an accuracy of only 0.2810, while the GAGNN-GIN model effectively failed to learn, halting its convergence early with an accuracy of 0.1187.

There are several factors that could explain why the GAGNN models failed in this scenario.

First and foremost is the fact that the GNN, whether it's a GAT or a GIN, shares the same weights for both group level and node level prediction. Since after the grouping stage nodes are pooled and aggregated together, the task of predicting what pattern generated that group becomes very

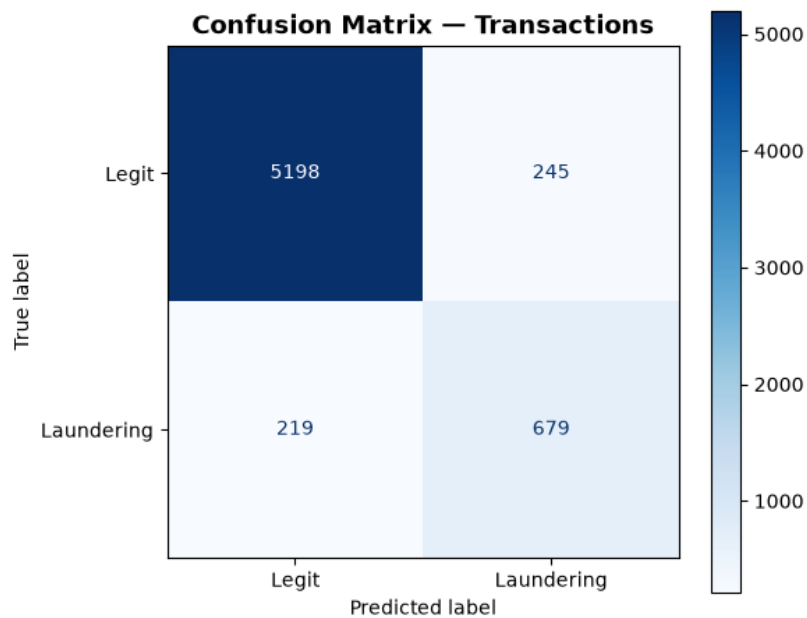


Figure 15: Confusion matrix for GAGNN-GAT (Binary task)

```
=====
GAGNN-GAT EVALUATION REPORT (Binary)
=====
```

Accuracy	: 0.9268
ROC-AUC	: 0.9065

```
-----
```

	precision	recall	f1-score	support
Legit	0.96	0.95	0.96	5443
Laundering	0.73	0.76	0.75	898
accuracy			0.93	6341
macro avg	0.85	0.86	0.85	6341
weighted avg	0.93	0.93	0.93	6341

Figure 16: Evaluation report for GAGNN-GAT (Binary task)

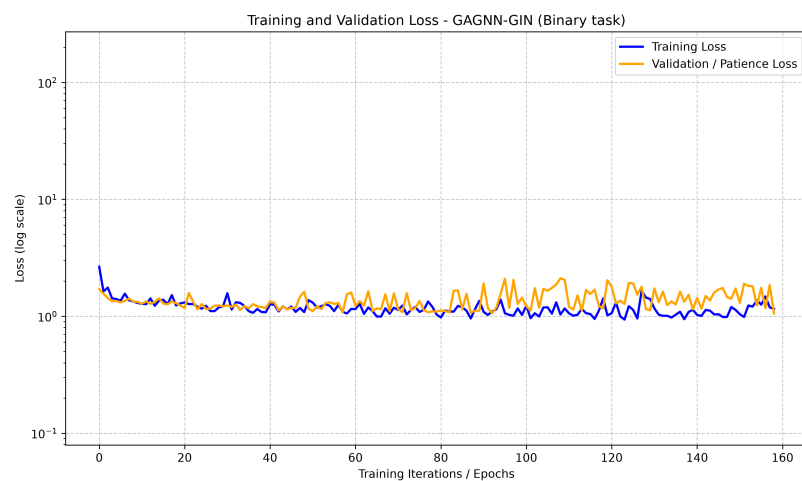


Figure 17: Training and patience loss for GAGNN-GIN (Binary task)

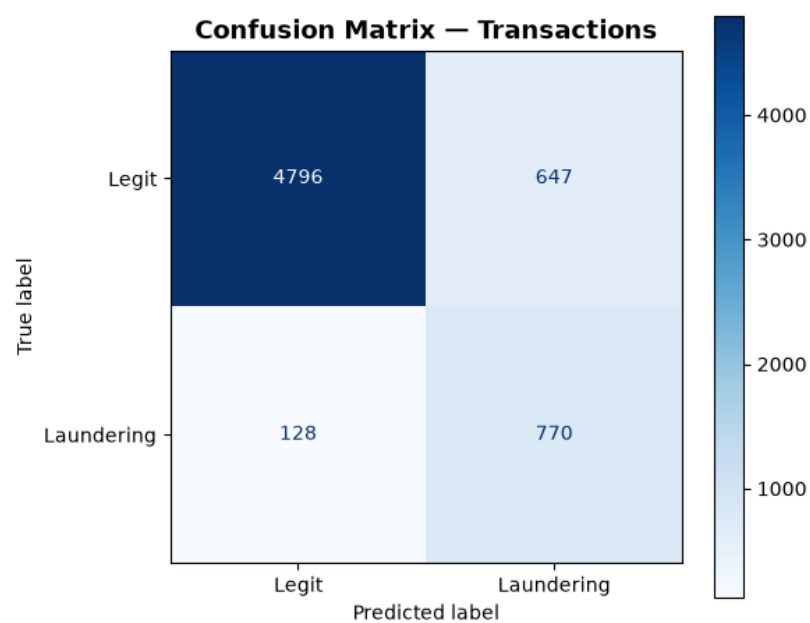


Figure 18: Confusion matrix for GAGNN-GIN (Binary task)

```

=====
GAGNN-GIN EVALUATION REPORT (Binary)
=====
Accuracy   : 0.8778
ROC-AUC    : 0.9025
-----

```

	precision	recall	f1-score	support
Legit	0.97	0.88	0.93	5443
Laundering	0.54	0.86	0.67	898
accuracy			0.88	6341
macro avg	0.76	0.87	0.80	6341
weighted avg	0.91	0.88	0.89	6341

Figure 19: Evaluation report for GAGNN-GIN (Binary task)

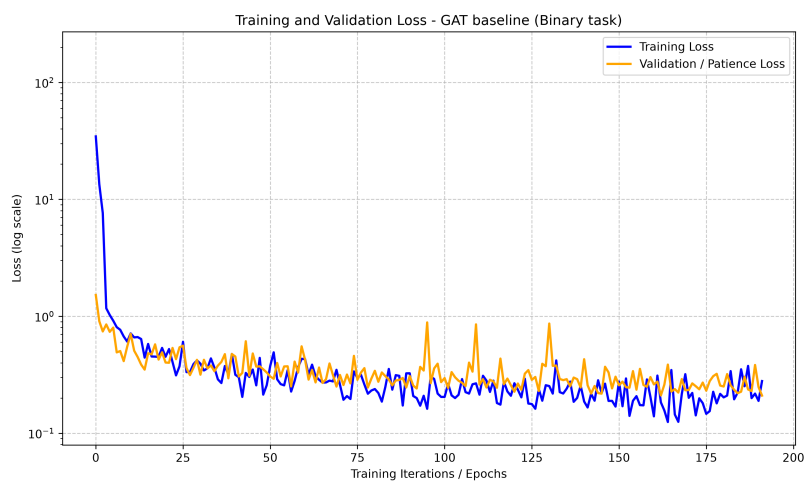


Figure 20: Training and patience loss for GAT baseline (Binary task)

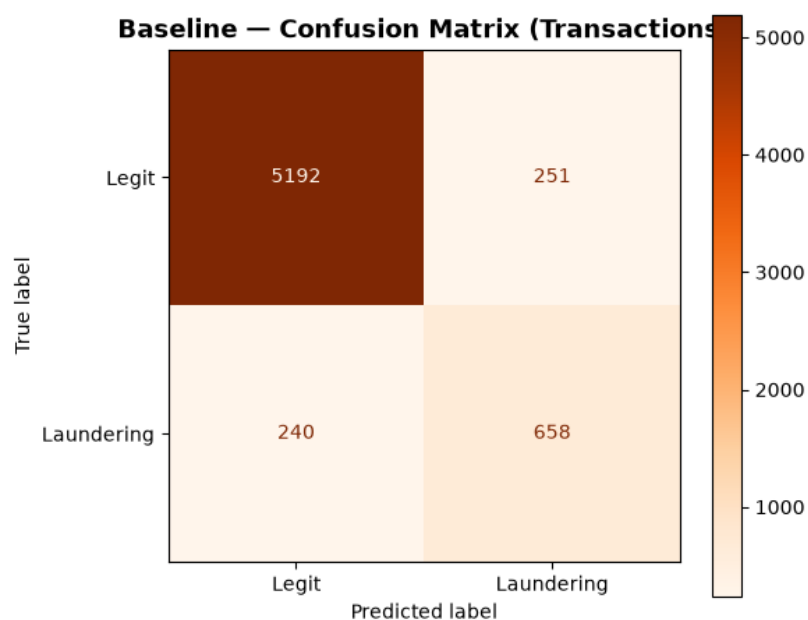


Figure 21: Confusion matrix for GAT baseline (Binary task)

=====				
GAT EVALUATION REPORT (Binary)				
=====				
Accuracy	: 0.9226			
ROC-AUC	: 0.9053			

	precision	recall	f1-score	support
Legit	0.96	0.95	0.95	5443
Laundering	0.72	0.73	0.73	898
accuracy			0.92	6341
macro avg	0.84	0.84	0.84	6341
weighted avg	0.92	0.92	0.92	6341

Figure 22: Evaluation report for GAT baseline (Binary task)

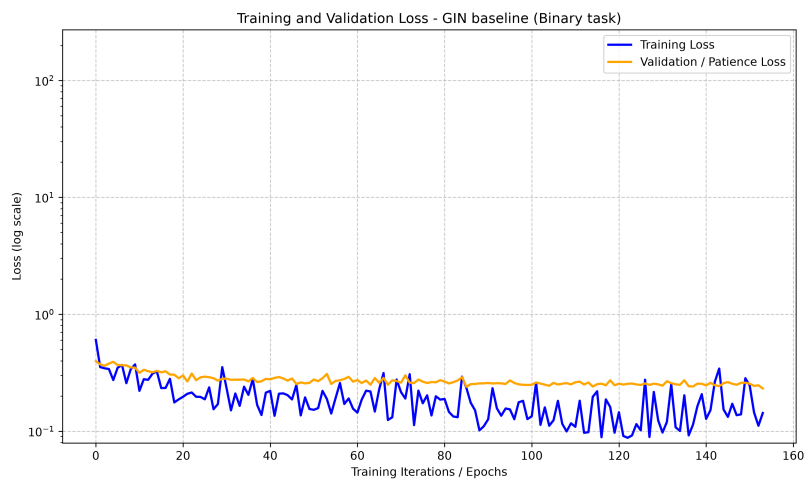


Figure 23: Training and patience loss for GIN baseline (Binary task)

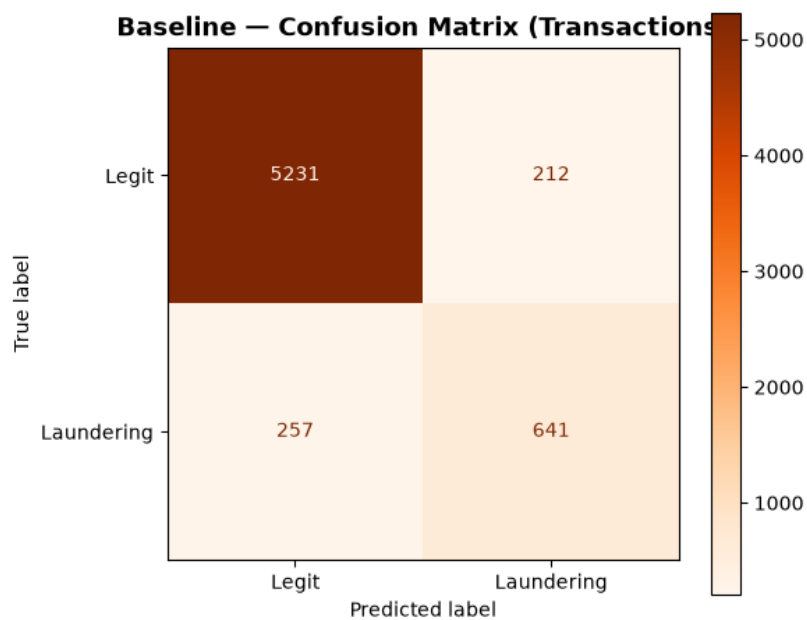


Figure 24: Confusion matrix for GIN baseline (Binary task)

GIN EVALUATION REPORT (Binary)				
Accuracy	: 0.9260			
ROC-AUC	: 0.8889			
	precision	recall	f1-score	support
Legit	0.95	0.96	0.96	5443
Laundering	0.75	0.71	0.73	898
accuracy			0.93	6341
macro avg	0.85	0.84	0.84	6341
weighted avg	0.92	0.93	0.93	6341

Figure 25: Evaluation report for GIN baseline (Binary task)

difficult for the GNN layers. While it is true that $\mathcal{L}_{\text{groups}}$ and $\mathcal{L}_{\text{nodes}}$ both are BCE even in the multiclass task, there is a high probability that the group level loss actively prevents the model from learning the right patterns. Also in this case future work regarding a wider exploration of the Hypothesis Space $\mathcal{H}$ by trying more values for c_1 could better explain this phenomenon.

The second major factor is the extreme data imbalance combined with the small size of the dataset. While the binary task simply grouped all anomalous topologies into a single *Laundering* class, the multiclass task splits them into eight distinct laundering patterns (e.g., *CYCLE*, *FAN-IN*, *BIPARTITE*, ...). Since the GAGNN architectures introduce a significantly higher number of trainable parameters and a much more complex optimization landscape compared to standard GNNs, when forced to learn complex decision boundaries for 9 different classes with such limited data, the model is likely to suffer from severe overfitting and often fails to generalize. The simpler baseline models, having fewer parameters, were slightly more robust to this lack of data. This doesn't necessarily mean that the GAGNN architectures are not suitable for this task but it's clear that a bigger dataset is necessary to properly leverage its architectural complexity.

Regarding the failure to converge for the GAGNN-GIN model, this can also be blamed on the assumption that optimal values of c_1, c_2 and c_3 found for the GAT model can be directly translated to the GIN model. The two architectures, albeit very similar, could work better with different degrees of importance assigned to the different loss components.

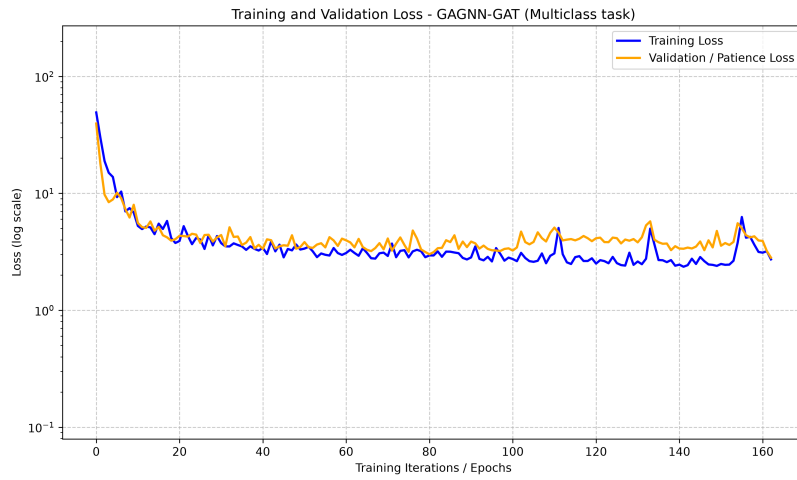


Figure 26: Training and patience loss for GAGNN-GAT (Multiclass task)

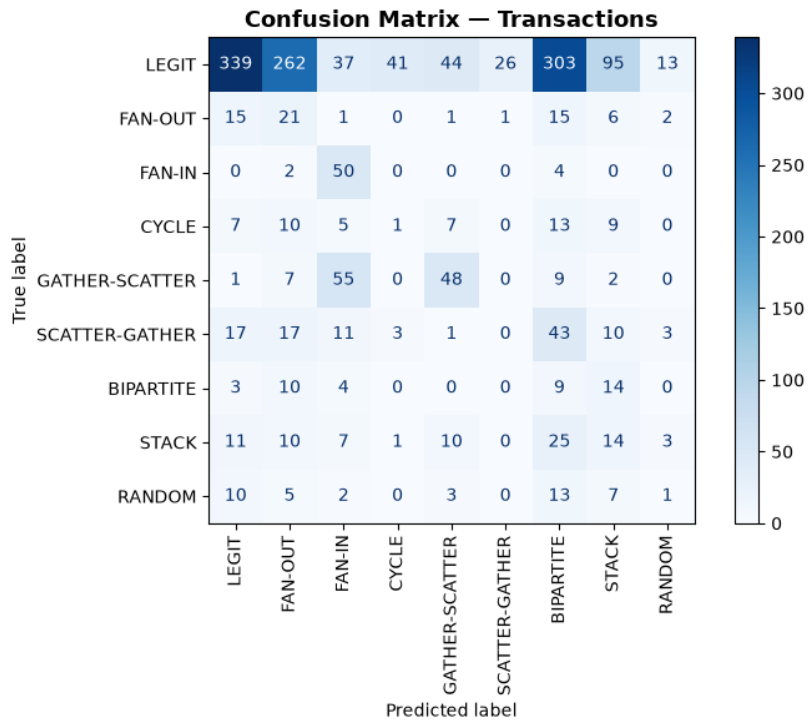


Figure 27: Confusion matrix for GAGNN-GAT (Multiclass task)

GAGNN-GAT EVALUATION REPORT (Multiclass)				
=====				
Accuracy : 0.2810				
ROC-AUC : 0.6639				

	precision	recall	f1-score	support
LEGIT	0.84	0.29	0.43	1160
FAN-OUT	0.06	0.34	0.10	62
FAN-IN	0.29	0.89	0.44	56
CYCLE	0.02	0.02	0.02	52
GATHER-SCATTER	0.42	0.39	0.41	122
SCATTER-GATHER	0.00	0.00	0.00	105
BIPARTITE	0.02	0.23	0.04	40
STACK	0.09	0.17	0.12	81
RANDOM	0.05	0.02	0.03	41
accuracy			0.28	1719
macro avg	0.20	0.26	0.18	1719
weighted avg	0.62	0.28	0.35	1719

Figure 28: Evaluation report for GAGNN-GAT (Multiclass task)

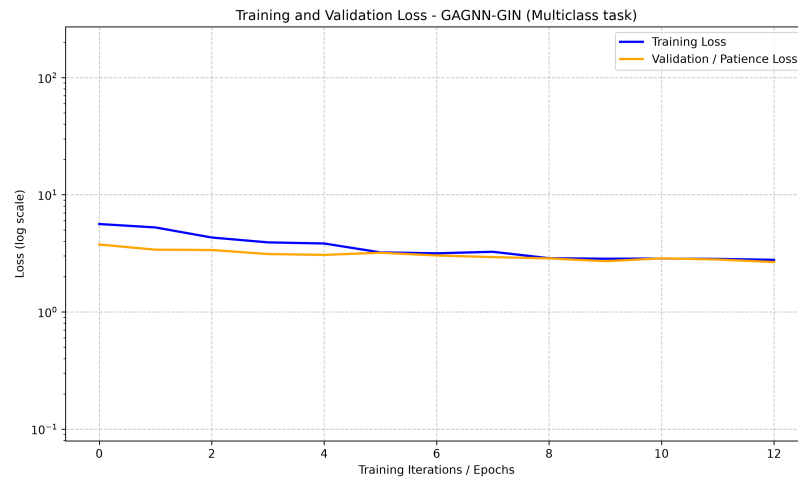


Figure 29: Training and patience loss for GAGNN-GIN (Multiclass task)

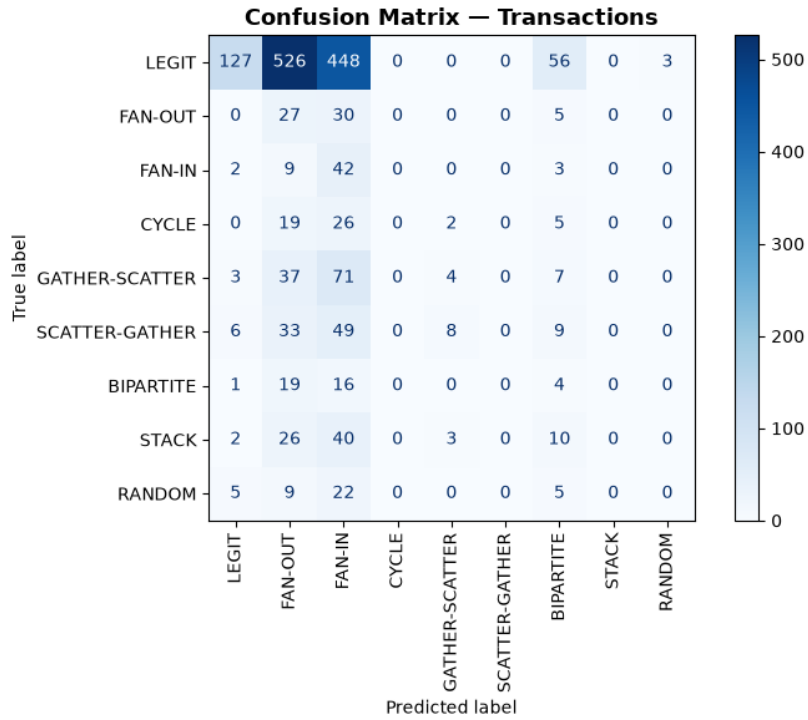


Figure 30: Confusion matrix for GAGNN-GIN (Multiclass task)

GAGNN-GIN EVALUATION REPORT (Multiclass)					
Accuracy : 0.1187					
ROC-AUC : 0.5777					
	precision	recall	f1-score	support	
LEGIT	0.87	0.11	0.19	1160	
FAN-OUT	0.04	0.44	0.07	62	
FAN-IN	0.06	0.75	0.10	56	
CYCLE	0.00	0.00	0.00	52	
GATHER-SCATTER	0.24	0.03	0.06	122	
SCATTER-GATHER	0.00	0.00	0.00	105	
BIPARTITE	0.04	0.10	0.06	40	
STACK	0.00	0.00	0.00	81	
RANDOM	0.00	0.00	0.00	41	
accuracy			0.12	1719	
macro avg	0.14	0.16	0.05	1719	
weighted avg	0.61	0.12	0.14	1719	

Figure 31: Evaluation report for GAGNN-GIN (Multiclass task)

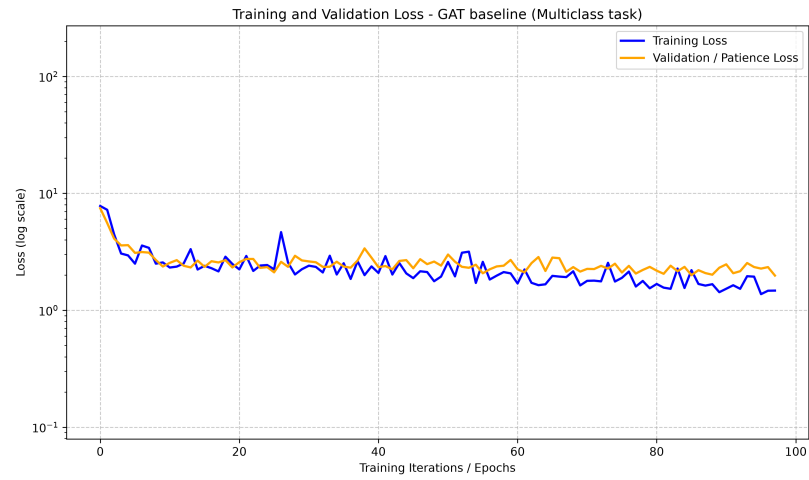


Figure 32: Training and patience loss for GAT baseline (Multiclass task)

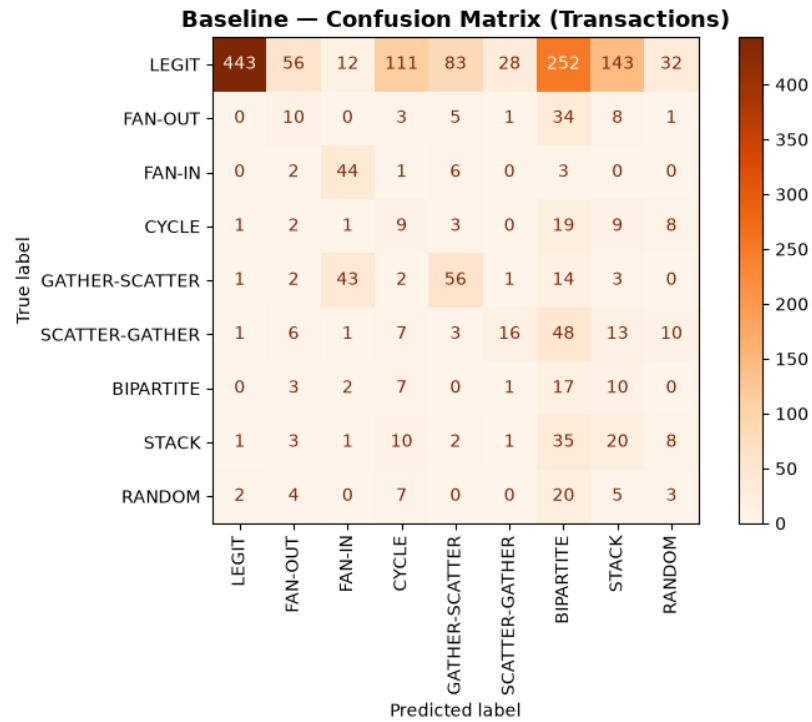


Figure 33: Confusion matrix for GAT baseline (Multiclass task)

GAT EVALUATION REPORT (Multiclass)				
=====				
Accuracy	: 0.3595			
ROC-AUC	: 0.7104			

	precision	recall	f1-score	support
LEGIT	0.99	0.38	0.55	1160
FAN-OUT	0.11	0.16	0.13	62
FAN-IN	0.42	0.79	0.55	56
CYCLE	0.06	0.17	0.09	52
GATHER-SCATTER	0.35	0.46	0.40	122
SCATTER-GATHER	0.33	0.15	0.21	105
BIPARTITE	0.04	0.42	0.07	40
STACK	0.09	0.25	0.14	81
RANDOM	0.05	0.07	0.06	41
accuracy			0.36	1719
macro avg	0.27	0.32	0.24	1719
weighted avg	0.74	0.36	0.45	1719

Figure 34: Evaluation report for GAT baseline (Multiclass task)

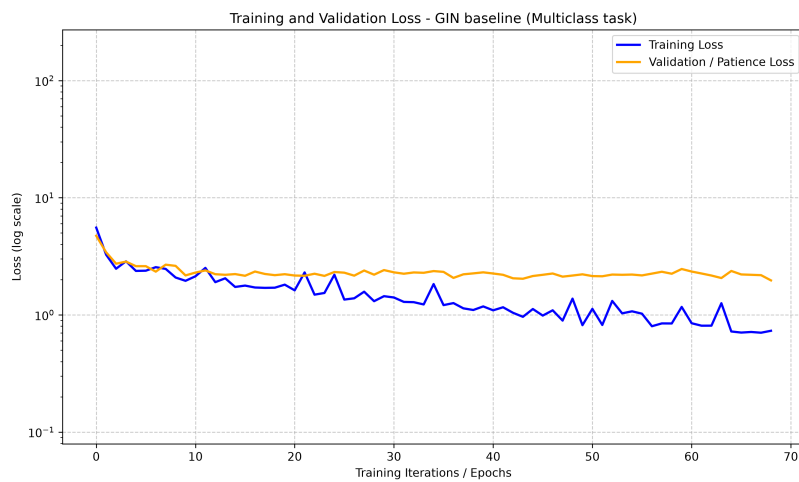


Figure 35: Training and patience loss for GIN baseline (Multiclass task)

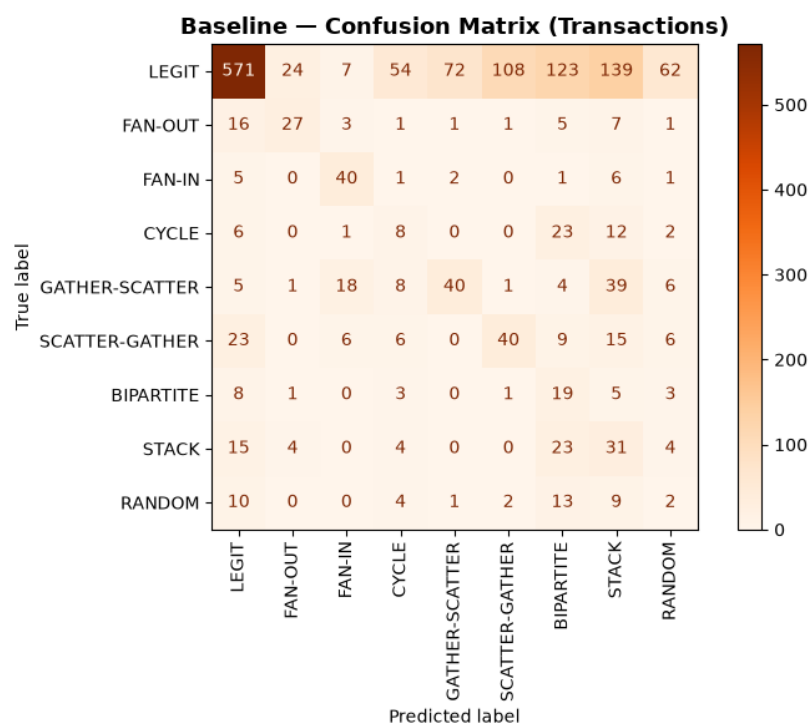


Figure 36: Confusion matrix for GIN baseline (Multiclass task)

```
=====
GIN EVALUATION REPORT (Multiclass)
=====
```

Accuracy	: 0.4526
ROC-AUC	: 0.7408

```
-----
```

	precision	recall	f1-score	support
LEGIT	0.87	0.49	0.63	1160
FAN-OUT	0.47	0.44	0.45	62
FAN-IN	0.53	0.71	0.61	56
CYCLE	0.09	0.15	0.11	52
GATHER-SCATTER	0.34	0.33	0.34	122
SCATTER-GATHER	0.26	0.38	0.31	105
BIPARTITE	0.09	0.47	0.15	40
STACK	0.12	0.38	0.18	81
RANDOM	0.02	0.05	0.03	41
accuracy			0.45	1719
macro avg	0.31	0.38	0.31	1719
weighted avg	0.67	0.45	0.52	1719

Figure 37: Evaluation report for GIN baseline (Multiclass task)

4.3 LIMITATIONS

Here we resume the limitations encountered during the study. Most of them are related to the poor hardware resources that we had available during this work.

The first major limitation is the fact that we didn't explore the hypothesis space of the parameters as much as we would have liked to. The assumption that c_1, c_2, c_3 translate directly from GAT to GIN, didn't always hold as we discovered during the experimentation. Furthermore the value of c_1 was kept fixed to 0.75 for all experiments, this allowed us to highly reduce the model selection computational cost, but it is possible that, especially for the multiclass task where from a theoretical analysis we believe that the group loss should hold less importance, we could have found better configurations. On top of that for architectural hyperparameters we often found the optimal values at the edges of the explored space, this suggests that the true best settings could lie outside of it.

Another problem came with the small size of the dataset, with the binary task it worked fine but for the multiclass one the need of a bigger dataset, that would have allowed the training of bigger models without overfitting, is clearly visible in both the training and test results. This limitation is interconnected with the first one since a bigger dataset would probably allow to train bigger models and thus we would have to further expand the hypothesis space.

Finally, limiting the number of neighbors during the training of the models to 15, 10 and 10 for the first three layers respectively (`num_neighbors = [15,10,10]`), in order to reduce the computational cost, introduced further noise in the loss computation and it's probably the reason why in some experiments the training loss is not as smooth as we expect it to be. Furthermore the fact that a fraudulent neighbor could not be sampled amongst the 15 ones the detection of the fraudulent community could be seriously compromised.

5

CONCLUSIONS

5.1 FUTURE DIRECTIONS

BIBLIOGRAPHY

- [1] Samuel Aidoo. Evaluating the effectiveness of aml regulations: A critical review. *AML/CFT Journal The Journal of Anti Money Laundering and Countering the Financing of Terrorism*, 07 2025. (Cited on pages 12 and 13.)
- [2] Erik Altman, Jovan Blanus, Luc Niederhäuser, Beni Egressy, Andreea Anghel, and Kubilay Atas. Realistic synthetic financial transactions for anti-money laundering models, 12 2023. (Cited on pages 3, 11, 14, 30, and 45.)
- [3] Canadian Anti-Fraud Centre. Canadian anti-fraud centre. <https://antifraudcentre-centreantifraude.ca/index-eng.htm>, 2026. Accessed: 2026-08-17. (Cited on page 7.)
- [4] Ana Isabel Canhoto. Leveraging machine learning in the global fight against money laundering and terrorism financing: An affordances perspective. *Journal of Business Research*, 131:441–452, 2021. (Cited on page 15.)
- [5] Dawei Cheng, Yujia Ye, Sheng Xiang, Zhenwei Ma, Ying Zhang, and Changjun Jiang. Anti-money laundering by group-aware deep graph learning. *IEEE Transactions on Knowledge and Data Engineering*, 35(12):12444–12457, 2023. (Cited on pages 7, 15, 23, 25, 39, 42, 46, and 47.)
- [6] Dawei Cheng, Yao Zou, Sheng Xiang, and Changjun Jiang. Graph neural networks for financial fraud detection: a review. *Frontiers of Computer Science*, 19(9), January 2025. (Cited on pages 19 and 23.)
- [7] Alfredo Contreras and Edgar Villa Pérez. Strategic choices in money laundering: Smurfing, layering, and financial over-diversification. *Journal of Economic Criminology*, 11:100199, 2026. (Cited on pages 9 and 10.)
- [8] Mariano-Florentino Cuéllar. The tenuous relationship between the knowing act of money laundering and its underlying crimes. *Columbia Law Review*, 103(2):312–431, 2003. (Cited on page 16.)

- [9] Europol. Does crime still pay? criminal asset recovery in the eu. Technical report, Europol, 2016. (Cited on page 7.)
- [10] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. (Cited on page 40.)
- [11] William L. Hamilton. *Graph Representation Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2020. (Cited on page 17.)
- [12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. (Cited on page 39.)
- [13] Nils M. Kriege, Fredrik D. Johansson, and Christopher Morris. A survey on graph kernels. *Applied Network Science*, 5(1):6, 2020. (Cited on page 17.)
- [14] Ouren Kuiper, Martin van den Berg, Joost van der Burgt, and Stefan Leijnen. Exploring explainable ai in the financial sector: Perspectives of banks and supervisory authorities. In *Benelux Conference on Artificial Intelligence*, pages 253–268. Springer, 2021. (Cited on page 15.)
- [15] Xiaoqi Li, Wenkai Li, Zhijie Liu, Meikang Qiu, Zhiquan Liu, Sen Nie, Zongwei Li, Shi Wu, and Yuqing Zhang. Scamsweeper: Detecting illegal accounts in web3 scams via transactions analysis. *IEEE Transactions on Information Forensics and Security*, 2025. (Cited on page 10.)
- [16] Adetoyese Omoseebi, Godwin Ola, and Jackson Tyler. Rule-based systems in aml, 03 2025. (Cited on pages 12 and 15.)
- [17] K. Singh and P. Best. Anti-money laundering: Using data visualization to identify suspicious activity. *International Journal of Accounting Information Systems*, 34:100418, 2019. (Cited on page 18.)
- [18] Toyotaro Suzumura and Hiroki Kanezashi. Anti-Money Laundering Datasets: InPlusLab anti-money laundering datadatasets. <http://github.com/IBM/AMLSim/>, 2021. (Cited on page 11.)
- [19] United Nations Office on Drugs and Crime (UNODC). *Estimating illicit financial flows resulting from drug trafficking and other transnational organized crimes*. UNODC Vienna, 2011. (Cited on page 7.)

- [20] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018. (Cited on page 19.)
- [21] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S. Yu Philip. A comprehensive study of graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. (Cited on page 18.)
- [22] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. (Cited on pages 20 and 21.)
- [23] Johan Östman, Edvin Callisen, Anton Chen, Kristiina Ausmees, Emanuel Gårdh, Jovan Zamac, Jolanta Goldsteine, Hugo Wefer, Simon Whelan, and Markus Reimegård. Amlgentex: Mobilizing data-driven research to combat money laundering, 2025. (Cited on page 11.)